

TMF630 REST API Design Guidelines Part 7: JSON Schemas guideline

TMF630 Part 7

Maturity Level: General Availability (GA)	Team Approved Date: September 11th, 2025
Release Status: Pre-production	Approval status: Team Approved
Version: 5.0.0	IPR Mode: RAND

Notice

Copyright © TM Forum 2025. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the [TM FORUM IPR Policy](#), must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an “AS IS” basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Direct inquiries to the TM Forum office:

100 Enterprise Drive
Suite 301 #1649
Rockaway, NJ 070866, USA
Tel No. +1 862 227 1648

TM Forum Web Page: www.tmforum.org

Table of Contents

Notice	2
Executive Summary	5
Conventions	6
Key words	6
Binding nature	6
Specification Standard	6
Formatting Conventions for Examples	7
Placeholders	7
Notation convention	7
1. Abstract base schemas	10
1.1. Addressable schema	10
1.2. Extensible schema	12
1.3. Entity schema	13
1.4. EntityRef schema	14
2. Patterns for concrete entity schemas	17
2.1. Pattern for [Entity] schemas	17
2.2. Pattern for [Entity]Ref schemas	21
2.3. Pattern for [Entity]RefOrValue schemas	27
2.4. Pattern for Related[Entity] schemas	32
2.5. Pattern for Related[Entity]RefOrValue schemas	36
2.6. Pattern for [Entity]Relationship schemas	39
3. Patterns for concrete entity item schemas	45
3.1. Pattern for [Entity]Item schemas	46
3.2. Pattern for [Entity]ItemRef schemas	49
3.3. Pattern for Related[Entity]Item schemas	53
3.4. Pattern for [Entity]ItemRelationship patterns	56
4. Event related schemas and patterns	59
4.1. Event schema	60
4.2. Pattern for [Entity][Event] schemas	64
4.3. Pattern for [Entity][Event]Payload schemas	66
5. Additional structural schemas	68
5.1. Characteristic schema	68
5.2. CharacteristicSpecification schema	78

5.3. CharacteristicValueSpecification schema	83
5.4. ExternalIdentifier schema	92
6. Additional structural patterns	95
6.1. Enumeration pattern	95
7. Type-like schemas	98
7.1. Quantity	98
7.2. Money	98
7.3. TimePeriod	98
8. Patterns and Schemas defined elsewhere in the Guidelines	99
8.1. History	99
8.2. Version	99
8.3. ImportJob / ExportJob	99
8.4. TaskResource	99
8.5. Monitor	99
8.6. Iterator	100
8.7. Error	100
References	101
Administrative Appendix	103
Document History	103
Acknowledgments	103

Executive Summary

The “REST API Design Guidelines” document provides guidelines and design patterns used in designing TM Forum REST APIs. The document is organized in multiple parts as follows:

- **Part One:** Practical guidelines for RESTful APIs; naming, CRUD, filtering, tasks, history, notifications
- **Part Two:** Advanced guidelines for RESTful API polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- **Part Three:** Guidelines for extending TMF Open APIs with hypermedia support
- **Part Four:** Advanced guidelines for RESTful APIs; lifecycle management, common tasks
- **Part Five:** JSON Patch extension to manage arrays
- **Part Six:** JSON Path extension
- **Part Seven:** JSON Schema patterns
- **Part Eight:** API Specialization

This part (Part 7) describes patterns to be applied for JSON schema definition. Our objective is to build consistent, reusable, and easy to design schemas in order to have consistent APIs more easily usable and understandable for the TMF community. To achieve this goal, we have defined a set of JSON schema patterns that should be applied when designing API resources.

JSON schemas are defined for all API classes. TMF tooling will build API OAS files & User Guides from these schemas and for each API a YAML rules file that defines the actual endpoints and operations supported by the API.

Conventions

Key words

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 8174 [RFC8174].

Binding nature

The prescriptions defined in this guideline are binding for all future applications. However, they MAY be temporarily violated in existing V5 APIs and SHOULD be fully adhered to when an API is further developed.

For Version 5, the TMForum Open APIs have undergone a major migration from Swagger 2.0 to OpenAPI Specification 3.0.x (OAS3). This change made the use of additional constructs in particular in the range of polymorphism and inheritance possible, which were not available in Swagger 2.0. To not slow down the migration process, the way of applying the new features was developed in parallel to the migration of the actual APIs, so that not all APIs were able to apply the new features in the same way. Likewise, TMF tooling was advances only stepwise to the new structures. Resultingly, not all V5 APIs are fully compliant with the prescriptions defined in this document.

In this document, we define the guidelines how APIs SHOULD be designed and temporarily accept historic deviations. For each pattern, the intended rules are stated, and if existing deviations are found in already published Version 5 APIs, they described as well. The intention is to apply the rules in future APIs and to migrate existing APIs to the new patterns at latest in the next major release. Breaking changes between minor releases of APIs within the Version 5 should be avoided where possible.

Specification Standard

The Version 5 TMForum Open APIs are currently based on the version 3.0.1 of the [OpenAPI Specification](#) standard. That standard makes use of the Draft 5 of the [JSON Schema](#) standard and its associated [JSON Schema Validation](#) standard.

In contrast, the TMF JSON Schemas built as reusable components for the TMF Open APIs are based on the newer Draft 7 of the [JSON Schema](#) standard and its associated [JSON Schema Validation](#) standard, which improve the resolution process for [\\$ref](#) references. The TMF tooling is able to process these schemas and generate valid OAS3 files from them.

With the further evolvement of TMForum's API tooling, APIs released in future will be based

on newer versions of the OpenAPI Specification.

Formatting Conventions for Examples

In most examples, attributes occur that normally would be long UUID values; as simplification of the layout (avoidance of unnecessary line breaks), these values are shortened to a length of 4 characters only, followed by two dots:

`bd69..` instead of `bd694f23-627f-4553-a1df-7e79d62001ac`

For the same reason, in examples we abbreviate the leading part of URLs by a placeholder:

`{apiRoot}` instead of `https://mycsp.com/tmf-api`

Placeholders

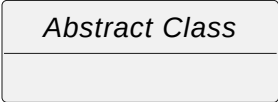
The reader should take note that

- When we use the notation **[Entity]**, it represents a placeholder to insert the name of a concrete entity that is managed as resource through a TMF Open API. If for example we have to use the **Related[Entity]** schema pattern for TroubleTicket then the schema's name will be **RelatedTroubleTicket**.
- When in a context of a specific **[Entity]** (with name starting with an upper case letter), we use the notation **[entity]** to denote the same name but starting with a lower case letter.
- The same convention applies to **[Event]**, **[event]**, and similar placeholders.
- All provided examples are illustrative and not systematically reflect an existing TMF schema.

Notation convention

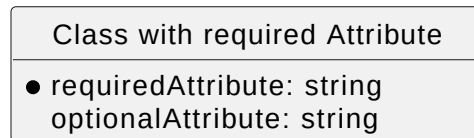
In this document, we use UML class diagrams to depict the structure of schemas. The following specific notation conventions are used:

- classes are named as the schema they represent
- class names printed italic indicate that the class is abstract, i.e. that it cannot be instantiated on itself, but only via a subclass.



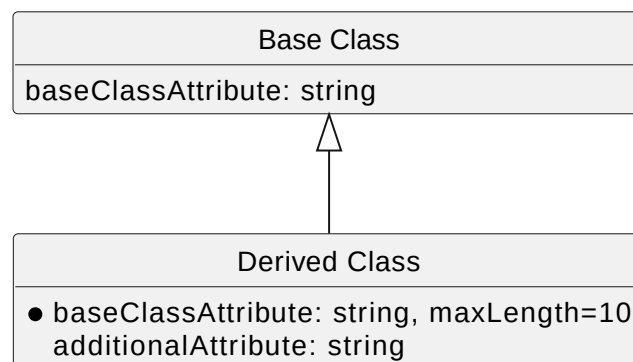
Abstract Class

- member names decorated with a bullet (•) indicate that the member is required, i.e. that instances of that class must have a valid value for that member.



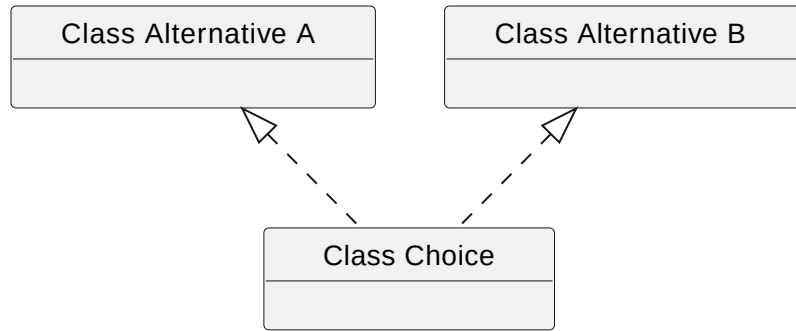
Note that this does not imply that for an implementation it is mandatory to implement the member; requirements on actual implementations are left to the API's conformance profile.

- a solid inheritance arrow between classes represents the OAS3 **allOf** relationship, i.e. that the schema represented by the class at the start of the arrow contains via **allOf** all members of the schema represented by the class at the end of the arrow, plus possibly its own additional members. An instance only validates against the first schema if it also validates against the second schema.

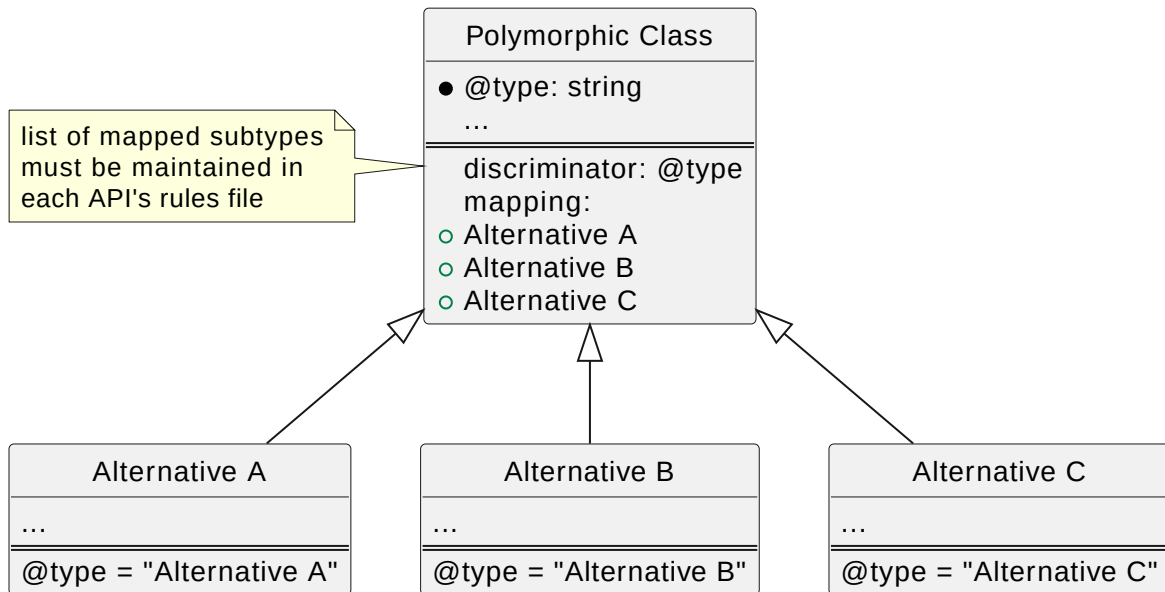


When a subclass has a member that already exists in the superclass, the subclass variant must have a compatible type and respect any imposed restrictions, but may further restrict the possible values, e.g. by defining it as required.

- dashed inheritance arrows from one class to several other classes indicate that the schema represented by the class at the start of the arrows is polymorphic via **oneOf**, i.e. that instances of the first class must validate against exactly one of the schemas represented by the classes at the end of the arrows, plus possibly its own additional members.



- where present, the lower compartment of a class symbol states additional restrictions on member values and/or values for discriminator properties, the latter marked by a circle.



Note that if Alternative A, ... are not directly or indirectly (by recursion) referred from the API's rules file, they have to be explicitly listed in that file in order for the tooling to generate the correct discriminator and mapping.

Chapter 1. Abstract base schemas

The following schemas are used as base for the actual entities, aiming at unification of the general structure and features applied throughout the TMF Open APIs.

They do not represent actual resources managed by APIs.



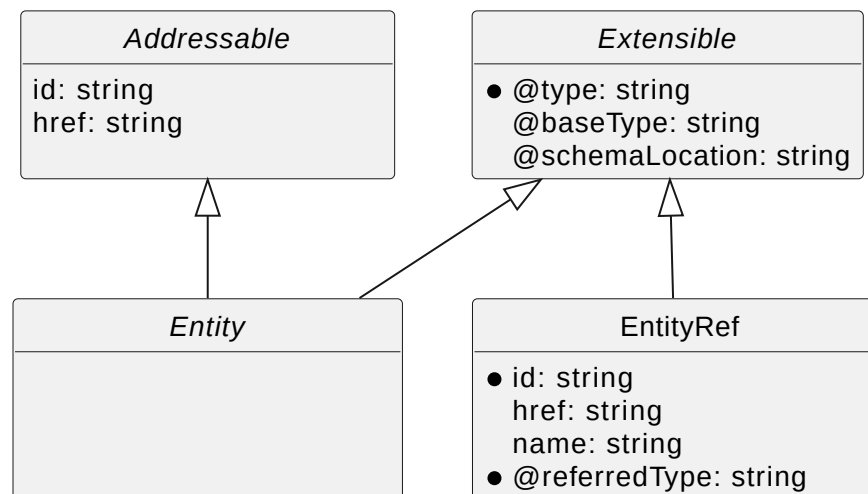
The term "abstract" is used here in the sense of "not directly instantiable", i.e. that these schemas cannot be used to define resources on their own, but only via subclasses.



In APIs that represent general purpose resources that can be applied in very different business contexts, like e.g. "Workflow", "Policy", "Intent", it may be necessary to include references to entities of unspecified type, because the actual kind of referred entity depends on the context. In such cases, the **EntityRef** schema can be used directly.

Overview

The following diagram shows the abstract base classes defined in the following sections:



1.1. Addressable schema

1.1.1. Purpose

This schema is used for any entity that could be addressable by an **id** and an **href**.

1.1.2. Definition

Addressable schema features only 2 attributes

- **id** – unique identifier of a resource
- **href** – hyperlink (uri or uri-reference) to access the resource

1.1.3. Rules

- **Addressable** is not directly used in any **[Entity]** schema but used as part of the **Entity** schema.
- This schema is defined in the Common domain and MUST NOT be changed in any API.

1.1.4. Schema

The schema file is `schemas/Tmf/Common/Addressable.schema.json`

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "Addressable.schema.json",
  "$privileged": true,
  "$comment": "Not to be modified",
  "title": "Addressable",
  "definitions": {
    "Addressable": {
      "$id": "#Addressable",
      "type": "object",
      "description": "Base schema for addressable entities",
      "properties": {
        "href": {
          "type": "string",
          "description": "Hyperlink reference"
        },
        "id": {
          "type": "string",
          "description": "unique identifier"
        }
      }
    }
  }
}
```

1.2. Extensible schema

1.2.1. Purpose

This schema is used for any schema that could leverage polymorphism pattern to be extended.

See [part 2 of the Guidelines](#) for more information on polymorphism.

1.2.2. Definition

Extensible schema features attributes

- **@type** – mandatory; this defines the **Extensible** sub-class name and is used for subclassing and as discriminator for polymorphism via **oneOf**.
- **@baseType** – When sub-classing, this defines the direct super-class.
- **@schemaLocation** – A URI to a JSON-Schema file that defines additional attributes and relationships. No prescription is made on the URI format.

1.2.3. Rules

- **Extensible** is not used directly in **[Entity]** schema but used as part of **Entity** or **EntityRef** schemas.
- This schema is defined in the Common domain and MUST NOT be changed in any API.
- **@type** is mandatory so that it can be used as discriminator for polymorphism via **oneOf**.

1.2.4. Schema

The schema file is **schemas/Tmf/Common/Extensible.schema.json**

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "Extensible.schema.json",
  "$privileged": true,
  "title": "Extensible",
  "definitions": {
    "Extensible": {
      "$id": "#Extensible",
      "type": "object",
      "description": "Base Extensible schema for use in TMForum Open-APIs - When used in a schema it means that the Entity described by the schema MUST be extended with the @type",
```

```

    "required": [ "@type" ],
    "properties": {
      "@type": {
        "type": "string",
        "description": "When sub-classing, this defines the sub-class
Extensible name"
      },
      "@baseType": {
        "type": "string",
        "description": "When sub-classing, this defines the super-class"
      },
      "@schemaLocation": {
        "type": "string",
        "description": "A URI to a JSON-Schema file that defines additional
attributes and relationships"
      }
    }
  }
}
}

```

1.3. Entity schema

1.3.1. Purpose

Entity schema MUST be used as base schema for any entity managed as resource of an TMF Open API, so that it becomes addressable with **id** and **href** and extensible through TMF polymorphic pattern.

1.3.2. Definition

Entity schema is a merge of **Addressable** and **Extensible** schemas.

1.3.3. Rules

- This schema is defined in the Common domain and MUST NOT be changed in any API.
- **Entity** schema MUST be used for any **[Entity]** schema via **allOf** (see **[Entity]** part).

1.3.4. Examples

Resources whose schemas extend the **Entity** schema:

- **Party**

- Product
- Service
- Resource
- ProductOrder
- ShoppingCart

1.3.5. Schema

The schema file is `schemas/Tmf/Common/Entity.schema.json`

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "Entity.schema.json",
  "$privileged": true,
  "title": "Entity",
  "definitions": {
    "Entity": {
      "$id": "#Entity",
      "type": "object",
      "description": "Base entity schema for use in TMForum Open-APIs.
Property.",
      "properties": {},
      "allOf": [
        { "$ref": "../Common/Addressable.schema.json#/definitions/Addressable" },
        { "$ref": "../Common/Extensible.schema.json#/definitions/Extensible" }
      ]
    }
  }
}
```

1.4. EntityRef schema

1.4.1. Purpose

`EntityRef` schema MUST be used by any schema describing a reference to an entity (examples: `ProductOrderRef`, `TroubleTicketRef`, `PartyRef`, etc...).

1.4.2. Definition

`EntityRef` schema is a merge of `Extensible` with additional attributes: the required `id`, `@referredType` and optional `href` and `name` attributes.



The 'id' and 'href' attributes do not refer to the **EntityRef** instance itself, but identify the referred entity instance. The **@referredType** attribute is used to indicate the type of the referred entity, which may be different from the type of the **EntityRef** itself.

1.4.3. Rules

- This schema is defined in the Common domain and MUST NOT be changed in any API.
- **EntityRef** schema must be used for **[Entity]Ref** schema via **allOf** (see **[Entity]Ref** part).
- This schema MAY only be used directly when a reference to an unspecified resource is needed, where the type of the referred entity is not known at design time.

1.4.4. Schema

The schema file is **schemas/Tmf/Common/EntityRef.schema.json**

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "EntityRef.schema.json",
  "$privileged": true,
  "title": "EntityRef",
  "definitions": {
    "EntityRef": {
      "$id": "#EntityRef",
      "description": "Entity reference schema to be use for all entityRef class.",
      "type": "object",
      "properties": {
        "id": {
          "type": "string",
          "description": "Unique identifier of the referred entity."
        },
        "href": {
          "type": "string",
          "format": "uri",
          "description": "Hyperlink reference to the referred entity."
        },
        "name": {
          "type": "string",
          "description": "Name of the referred entity."
        }
      }
    }
  }
}
```

```
    "@referredType": {
      "type": "string",
      "description": "Type of the referred entity."
    },
    "required": [ "id", "@referredType" ],
    "allOf": [
      { "$ref": "../Common/Extensible.schema.json#/definitions/Extensible" }
    ]
  }
}
```

1.4.5. Historic Deviations

In the current version 5 of the TMF Open APIs, `EntityRef` schema also uses `Addressable` via `allOf`, to inherit its `id` and `href` attributes.

`EntityRef` does not represent something that can be addressed, but instead is used to establish a reference to an existing entity.

As such, this inheritance does not reflect semantics of `EntityRef`, but is a pure implementation inheritance.

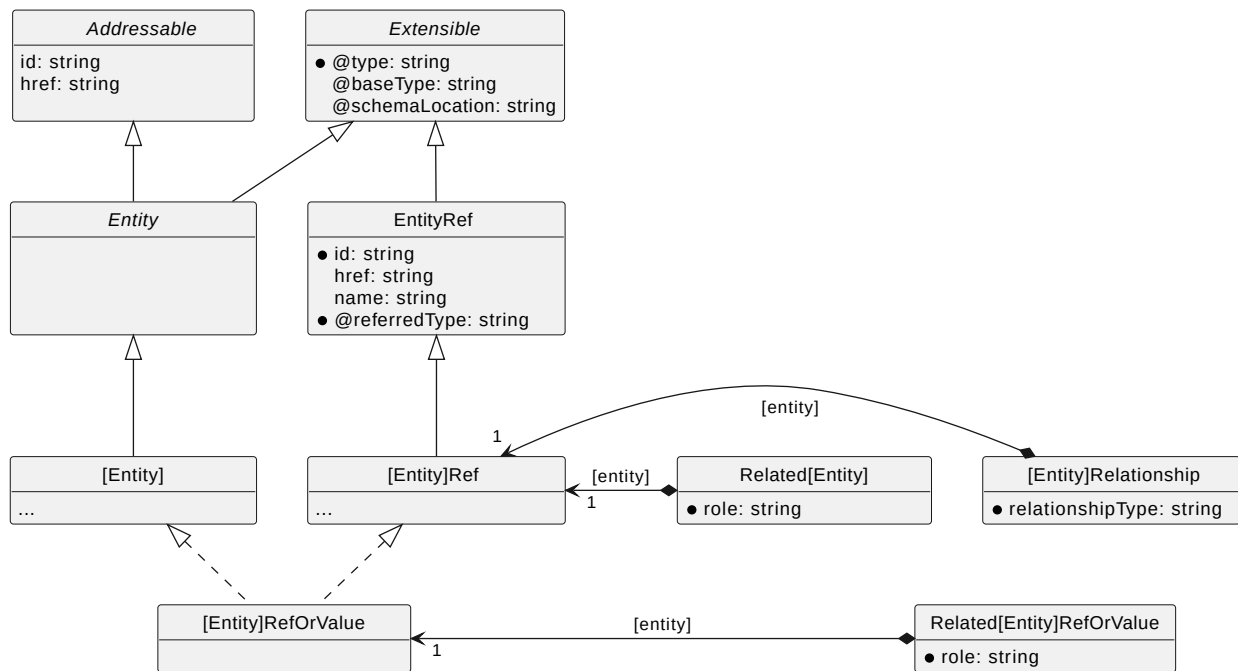
This will be corrected in future releases, where `EntityRef` will not inherit from `Addressable` anymore but carry the `id` and `href` attributes on its own, as is already the case (and overloading `id` to be required) and is shown in the above diagram and schema.

Chapter 2. Patterns for concrete entity schemas

This chapter specifies patterns for building concrete entity related schemas.

Overview

The following diagram shows an overview on the patterns for concrete entity schemas:



The patterns `[Entity]Relationship`, `Related[Entity]` and `Related[Entity]RefOrValue` have been changed in V5 to use the `[Entity]Ref` pattern instead of either including the referring attributes `id`, `href` and `@referredType` directly or inheriting them from `EntityRef`, as was the case in prior versions.

2.1. Pattern for [Entity] schemas

2.1.1. Purpose

The `[Entity]` schema is used to describe any API resource representation. An `[Entity]` SHOULD be managed as a resource by one and only one of the TMF APIs.

Examples: `TroubleTicket`, `Service`, `ProductOrder`, `ShoppingCart`, `Quote`, etc...



This pattern cannot be applied to the entity **Entity** of unspecified type, as this would collide with the basic **Entity** schema.

2.1.2. Definition

[**Entity**] schema is a merge of **Entity** schema and additional attributes.

2.1.3. Rules

- MUST include a direct or indirect (via recursion) reference to **Entity** subschema (use of **allOf** keyword in OpenAPI Specification 3.0.x (OAS)).
- SHOULD contain appropriate attributes of any of the basic datatypes **string**, **integer**, **number**, **boolean**, or sub-resources (JSON type **object**, more specifically defined by a schema), or arrays of such, used to describe the [**Entity**] class.

For specific attributes, the following additional rules apply:

- If the resource needs a **type** attribute the attribute name MUST NOT be named **type** but **[entity]Type**.
- If the resource requires a **timestamp** to manage the date and time when the resource is created, this attribute MUST be named **creationDate** and be of type **string** with format **date-time**.
- If the resource requires a **validity period** to manage the time period for which the resource is valid, this attribute MUST be named **validFor** and MUST be a reference to class **TimePeriod**.
- If the resource requires some **quantity value** to represent any property measured as integer or fractional multiple of some unit of measure, the corresponding attribute MUST be a reference to class **Quantity**.
- If the resource requires some **monetary value**, the corresponding attribute MUST be a reference to class **Money**.
- If the resource requires some **state value** to represent its condition within an operational process mainly controlled by automation, in particular if being prescriptive for what the object is allowed or able to do, having standardized transitions and business rules that can be formally described by a state machine, the attribute's name MUST either be **state** or end on **State** and be either of type **string** which MAY be restricted by specifying the available values as **enum**, or a reference to an enumeration class.
- If the resource requires some **status value** to represent its stage in a business process, where the advancement depends on managerial business decisions, in particular when

being descriptive as operational indicator or metric or reporting how well the object is currently doing, the corresponding attribute's name MUST either be **status** or end on **Status** and be either of type **string** which MAY be restricted by specifying the available values as **enum**, or a reference to an enumeration class.



The above distinction between **state** and **status** is not always clear-cut, and in some cases the terms are used interchangeably. The intention of the above rules is to establish a consistent usage throughout all TMF Open APIs and across other SDOs (Standards Developing Organization). Existing APIs may not yet fully comply with this intention, but it should be applied in future APIs and existing APIs should be aligned with this intention in future major releases.

2.1.4. Examples

Example ShoppingCart schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ShoppingCart.schema.json",
  "title": "ShoppingCart",
  "definitions": {
    "ShoppingCart": {
      "$id": "#ShoppingCart",
      "description": "Shopping Cart resource is used for the temporarily
selection and reservation of product offerings in e-commerce, call center and
retail purchase. Shopping cart supports purchase of both physical and digital
goods and service (e.g. handset, telecom network service). Shopping Cart contain
list of cart items, a reference to customer (partyRole) or contact medium in case
customer not exist, and the total items price including promotions",
      "type": "object",
      "properties": {
        "validFor": {
          "$ref": "../Common/TimePeriod.schema.json#/definitions/TimePeriod",
          "description": "The period for which the shopping cart is valid (e.g.
90 if no activity or 7 days if cart is empty)"
        },
        "contactMedium": {
          "type": "array",
          "items": {
            "$ref":
"../Common/ContactMedium.schema.json#/definitions/ContactMedium"
          }
        }
      }
    }
  }
}
```

```

    },
    "cartTotalPrice": {
      "type": "array",
      "items": {
        "$ref": "../Customer/CartPrice.schema.json#/definitions/CartPrice"
      },
      "description": "Total amount of the shopping cart, usually of money,
that represents the actual price paid by the Customer for cart (considering only
\"Active\" cart items)"
    },
    "cartItem": {
      "type": "array",
      "items": {
        "$ref": "../Customer/CartItem.schema.json#/definitions/CartItem"
      }
    },
    "relatedParty": {
      "type": "array",
      "items": {
        "$ref":
"../EngagedParty/RelatedPartyOrPartyRole.schema.json#/definitions/RelatedPartyOrP
artyRole"
      }
    },
    "creationDate": {
      "type": "string",
      "format": "date-time",
      "description": "The date on which the shopping cart was created"
    },
    "lastUpdate": {
      "type": "string",
      "format": "date-time",
      "description": "The date and time that the shopping cart was last
updated"
    },
    "allOf": [
      { "$ref": "../Common/Entity.schema.json#/definitions/Entity" }
    ]
  }
}

```

2.1.5. Historic Deviations

Redundant inheritances

In some cases, a schema that is defined as `allOf Entity` additionally inherits from `Addressable` or even `Extensible`. This is redundant, as `Entity` already inherits from `Addressable` and `EntityRef`.

Removing the redundant inheritance does not impact the structure defined by the resulting schema, and also not the APIs using this deviation. Removal does not create a breaking change for existing V5 APIs.

Inadequate EntityRef and Entity polymorphism

Some schemas are defined as `oneOf Entity and EntityRef` and match the `[Entity]RefOrValue` pattern, but their name does not follow the convention to reflect their nature.

Since this can lead to misunderstandings, these schemas should either be renamed to follow the convention, or the polymorphism should be adjusted so that the name reflects the nature of the schema (to be either an entity or a reference).

*Additional inheritance from EntityRef *

Some schemas are even defined as `allOf Entity and EntityRef`, which implies that objects of this type are entities and references at the same time.

This should be corrected depending on the purpose and naming of the schema to either inherit from only one of these schemas, or use `oneOf` to express the polymorphic nature.

Retundant attributes

Some `[Entity]` schemas (that are defined as `allOf Entity`) redundantly include attributes `id`, `href` and/or `@type` on their own, although they already inherit these attributes from `Entity`.

Removing them does not impact the structure defined by the resulting schema, and also not the APIs using this deviation. Removal does not create a breaking change for existing V5 APIs.

2.2. Pattern for [Entity]Ref schemas

2.2.1. Purpose

[Entity]Ref schema is used to describe a reference to an existing instance of [Entity] or any of its subclasses. This referred entity must be accessible via GET operation from an existing (or future) TMF API.



This pattern SHOULD only be used if the semantics of both the referred entity as well as the relation to the referred entity is clear from the context of the attribute using the [Entity]Ref schema. If this is not the case, either the Related[Entity] or the [Entity]Relation pattern SHOULD be used, to better indicate the semantics.

Example:

In the Product schema the following schemas are used to enable referring to corresponding entities: BillingAccountRef, ProductOfferingRef, ProductSpecificationRef, ResourceRef, ServiceRef.



This pattern cannot be applied to the entity Entity of unspecified type, as this would collide with the basic EntityRef schema.

2.2.2. Definition

[Entity]Ref schema is a merge of EntityRef schema and possibly selected attributes from [Entity] schema.

2.2.3. Rules

- MUST include a reference to EntityRef schema or any other [Entity]Ref directly or indirectly derived from EntityRef (by use of allof key word in OpenAPI Specification 3.0.x (OAS)).
- MAY include additional attributes from the [Entity] that are considered important.

Example:

ResourceRef schema features a value attribute. This represents the human-readable value associated with the resource, it might be any of:

- MSISDN
- Serial number
- IMSI
- etc

- The associated **[Entity]** schema MUST contain the **id** attribute, and its instances can only be referred to if this attribute is populated.

2.2.4. Examples

ServiceSpecificationRef schema



See the additional **version** attribute.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ServiceSpecificationRef.schema.json",
  "title": "ServiceSpecificationRef",
  "definitions": {
    "ServiceSpecificationRef": {
      "$id": "#ServiceSpecificationRef",
      "description": "reference to a Service Specification.",
      "type": "object",
      "properties": {
        "version": {
          "type": "string",
          "description": "Service Specification version"
        }
      },
      "allOf": [
        { "$ref": "../Common/EntityRef.schema.json#/definitions/EntityRef" }
      ]
    }
  }
}
```

Special case PartyRefOrPartyRoleRef schema:

In a lot of API like all order API, trouble ticket, quote, etc... we leave the choice to

- either link related **Party**(ies) to these resources and indicate as attribute the role played by the party with respect to the resource,
- or to use **PartyRole** entities instead. In that case, the **role** attribute of the **RelatedPartyOrPartyRole** MUST take the same value as the name attribute of the referred **PartyRole** instance.

For this, the **PartyRefOrPartyRoleRef** schema is used (see the **Related[Entity]** pattern),

which is via **oneOf** either a **PartyRef** or a **PartyRoleRef**:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "PartyRefOrPartyRoleRef.schema.json",
  "$privileged": true,
  "title": "PartyRefOrPartyRoleRef",
  "definitions": {
    "PartyRefOrPartyRoleRef": {
      "$id": "#PartyRefOrPartyRoleRef",
      "description": "",
      "type": "object",
      "oneOf": [
        { "$ref": "../EngagedParty/PartyRef.schema.json#/definitions/PartyRef" },
        { "$ref":
"../EngagedParty/PartyRoleRef.schema.json#/definitions/PartyRoleRef" }
      ]
    }
  }
}
```

with

*PartyRef schema: *

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "PartyRef.schema.json",
  "title": "PartyRef",
  "definitions": {
    "PartyRef": {
      "$id": "#PartyRef",
      "description": "A Party reference",
      "type": "object",
      "allOf": [
        { "$ref": "../Common/EntityRef.schema.json#/definitions/EntityRef" }
      ]
    }
  }
}
```

*PartyRoleRef schema: *

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "PartyRoleRef.schema.json",
  "title": "PartyRoleRef",
  "definitions": {
    "PartyRoleRef": {
      "$id": "#PartyRoleRef",
      "description": "Party role reference. A party role represents the part
played by a party in a given context.",
      "type": "object",
      "properties": {
        "partyId": {
          "type": "string",
          "description": "The identifier of the engaged party that is linked to
the PartyRole object."
        },
        "partyName": {
          "type": "string",
          "description": "The name of the engaged party that is linked to the
PartyRole object."
        }
      },
      "allOf": [
        { "$ref": "../Common/EntityRef.schema.json#/definitions/EntityRef" }
      ]
    }
  }
}
```



The additional attributes **partyId** and **partyName** are not directly taken from **PartyRole** as allowed by the pattern, but are attributes **id** and **name** of **PartyRole**'s attribute **engagedParty**. So when strictly following the pattern, **PartyRoleRef** should instead have the additional attribute **engagedParty** of type **PartyRef**, which then carried **id** and **name**. This deviation is accepted for the current V5 APIs, but might be adjusted in future releases.

2.2.5. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the

[overview diagram](#)):

Additional inheritance from Extensible

In addition to be `allof EntityRef`, which is is `allof Extensible`, some `[Entity]Ref` schemas redundantly are `allof Extensible` on their own. This does not harm, but is not needed.

Removing the redundant inheritance does not impact the structure defined by the resulting schema, and also not the APIs using this deviation. Removal does not create a breaking change for existing V5 APIs.

Additional inheritance from Entity

In some cases `[Entity]Ref` is not only `allof EntityRef`, but additionally `allof Entity`.

This adds inheritance from `Extensible`, which is redundant as explained in the previous case.

This also adds inheritance from `Addressable`, which does not reflect the semantics of `EntityRef` as explained in the [EntityRef schema](#) section.

Since `EntityRef` already contains the attributes `id` and `href` that these `[Entity]Ref` cases wrongly inherit from `Addressable`, this again does not impact the structure defined by the resulting schema, but is semantically incorrect.

Removing the redundant inheritance does not impact the structure defined by the resulting schema, and also not the APIs using this deviation, but better reflects the intended semantics. Removal does not create a breaking change for existing V5 APIs.

Additional attribute id and/or href and/or @type and/or @referredType

`[Entity]Ref` already contains these attributes inherited from `EntityRef`, so adding them again as own attributes is redundant. It does not harm, but is not needed.

Removing these redundant attribute does not impact the structure defined by the resulting schema, and also not the APIs using this deviation. Removal does not create a breaking change for existing V5 APIs.

Additional attributes not from [Entity]

`[Entity]Ref` may contain attributes from the related `[Entity]`, but in some cases further attributes are included that do not exist in the related `[Entity]` schema.

In most cases, the added attribute is `version` or `state`. The problem could possibly be resolved by adding these missing attributes to the corresponding `[Entity]` schema, if they

are considered relevant for that entity.

In other cases, the related `[Entity]` schema does not (yet) exist; it should be checked if these `[Entity]Ref` are necessary at all, or if the corresponding `[Entity]` schema should be created, probably with a corresponding API maintaining that kind of objects.

Depending on how the schemas will be modified, aligning them with the pattern may or may not create a breaking change for existing V5 APIs.

An exceptional case is `PartyRoleRef`, which includes attributes `partyId` and `partyName`, but this refers to the `id` and `name` attributes of the `Party` instance related to the `PartyRole` by its attribute `engagedParty` of type `PartyRef`. This double indirection is accepted as an exception for V5 APIs, but might be adjusted in future versions..

Not using `[Entity]Ref`

The pattern supports indirect inheritance of `[Entity]Ref` by defining it as `allOf` some other `[Entity]Ref` that itself follows the pattern. But in some cases, that intermediate resource does not follow the pattern.

Changing the intermediate resource to strictly follow the pattern also corrects the `[Entity]Ref` depending in it.

2.3. Pattern for `[Entity]RefOrValue` schemas

2.3.1. Purpose

This schema is used to describe **either** an existing entity via its reference (by ref) **or** literally an existing or not (yet) existing entity (by value).

It should be considered that `OrValue` feature MAY be added only when the value pattern is relevant in **providing** information **via POST** (and/or PATCH) operation.

This pattern MUST NOT be used for retrieving information in GET response (where EXPAND & DEPTH keywords could be leveraged).

In general, `[Entity]RefOrValue` schema SHOULD only be used to refer to entities managed in the same API as the referring entity, or if it must be assumed that the entity instance included by value has not yet been persisted via its own API.

A good example is provided in the Product Ordering Management API where an ordered Product could be provided by reference (to change or remove a Product already existing in the Product Inventory) or by value (a new one that is not yet stored in the inventory, to transport product intent).



Although not strictly following the naming convention, the

PartyOrPartyRole schema is an application of this pattern, as it is via **oneOf** a **PartyRef** or **PartyRoleRef** **OR** a **Party** or **PartyRole** or any of their subclasses. The formally correct name would be **PartyRefOrValueOrPartyRoleRefOrValue**.

When inclusion by value is not required, the **PartyRefOrPartyRoleRef** SHOULD be used instead, via **oneOf** it is only a **PartyRef** or **PartyRoleRef**.

2.3.2. Rules

- MUST be built from **[Entity]** schema and **[Entity]Ref** schema as alternatives (use of **oneOf** keyword in OpenAPI Specification 3.0.x (OAS)).
If **[Entity]** is an abstract class, all its concrete subclasses MAY be used instead of **[Entity]**, and **[Entity]Ref** MAY be replaced by references to the concrete subclasses.
- Attributes **@type**, **@baseType** and **@schemaLocation** **are related to the selected entity alternative** (i.e. the **[Entity]** or **[Entity]Ref**) and not the **[Entity]RefOrValue** itself.
- Attribute **@referredType** eventually inherited from **[Entity]Ref** is related to the referred entity, not the **[Entity]RefOrValue** or **[Entity]Ref**.
- MUST NOT include any additional attributes.

2.3.3. Examples

Example for **ProductRefOrValue**:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ProductRefOrValue.schema.json",
  "title": "ProductRefOrValue",
  "definitions": {
    "ProductRefOrValue": {
      "$id": "#ProductRefOrValue",
      "description": "A product to be created defined by value or existing  
defined by reference. The polymorphic attributes @type, @schemaLocation &  
@referredType are related to the product entity and not the ProductRefOrValue  
class itself",
      "type": "object",
      "properties": { },
      "oneOf": [
        { "$ref": "ProductRef.schema.json#/definitions/ProductRef" },
        { "$ref": "Product.schema.json#/definitions/Product" }
      ]
    }
  }
}
```

```
}
}
```



The TMF Tooling will use the list of alternatives to generate the corresponding discriminator property in the OpenAPI Specification 3.0.x (OAS) file, so that the API user can select the alternative to use. Where listed in the respective API's rules file, also specialized schemas (via [allOf](#)) of the [\[Entity\]](#) or [\[Entity\]Ref](#) schema will be considered for the discriminator's values and mapping.

Special case: PlaceRefOrValue schema:

This schema considers that [Place](#) is an abstract class, so that the alternatives for inclusion by value are its concrete subclasses:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "PlaceRefOrValue.schema.json",
  "title": "PlaceRefOrValue",
  "definitions": {
    "PlaceRefOrValue": {
      "$id": "#PlaceRefOrValue",
      "description": "The polymorphic attributes @type, @schemaLocation & @referredType are related to the Place entity and not the PlaceRefOrValue class itself",
      "type": "object",
      "oneOf": [
        { "$ref": "PlaceRef.schema.json#/definitions/PlaceRef" },
        { "$ref": "GeographicLocation.schema.json#/definitions/GeographicLocation" },
        { "$ref": "GeographicSite.schema.json#/definitions/GeographicSite" },
        { "$ref": "GeographicAddress.schema.json#/definitions/GeographicAddress" }
      ]
    }
  }
}
```



The abstract class [Place](#) is not included in that list, only its concrete subclasses.

Instantiation examples for PlaceRefOrValue:

In this case, references resp. values are provided as instances of subclasses of the abstract **Place**, here for **GeographicAddress**.

For Ref:

```
"place": {
  "@type" : "PlaceRef",
  "@referredType" : "GeographicAddress",
  "id" : "690574e8-4d5b-4fa2-b19f-6e812112b76a",
  "href": "https://mycsp.com:8080/tmf-api/geographicAddress/690574e8-4d5b-4fa2-b19f-6e812112b76a",
  "name" : "Name of the referred address in context of the resource containing this reference as sub-entity"
}
```

For Value:

```
"place": {
  "@type" : "GeographicAddress"
  "@schemaLocation" : "url where the GeographicAddress schema is described ...
not the placeRefOrValue schema"
  "name": "Jacob House"
  "streetNr": 4,
  "streetName": "Boisseau",
  "streetType": "rue",
  ...
}
```



The "Value" variant is used only in POST operations for an entity containing this **GeographicAddress** instance as sub-entity, and only if that instance has not yet been created via e.g. the Geographic Address Management API. Therefore, the instance does not contain a value for the **id** attribute, and also the **href** attribute is not present.

Special case: **PartyOrPartyRole** schema:



The naming of this schema does not strictly follow the naming convention for this pattern. This may lead to inadvertently include **Party/PartyRole** by value where this is not intended. The correct name should be **PartyRefOrValueOrPartyRoleRefOrValue**.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "PartyOrPartyRole.schema.json",
  "$privileged": true,
  "title": "PartyOrPartyRole",
  "definitions": {
    "PartyOrPartyRole": {
      "$id": "#PartyOrPartyRole",
      "description": "",
      "type": "object",
      "oneOf": [
        { "$ref": "../EngagedParty/PartyRef.schema.json#/definitions/PartyRef" },
        { "$ref":
"../EngagedParty/Individual.schema.json#/definitions/Individual" },
        { "$ref":
"../EngagedParty/Organization.schema.json#/definitions/Organization" },
        { "$ref":
"../EngagedParty/PartyRoleRef.schema.json#/definitions/PartyRoleRef" },
        { "$ref": "../EngagedParty/PartyRole.schema.json#/definitions/PartyRole"
      },
        { "$ref": "../EngagedParty/Supplier.schema.json#/definitions/Supplier" },
        { "$ref":
"../EngagedParty/BusinessPartner.schema.json#/definitions/BusinessPartner" },
        { "$ref": "../EngagedParty/Consumer.schema.json#/definitions/Consumer" },
        { "$ref": "../EngagedParty/Producer.schema.json#/definitions/Producer" }
      ]
    }
  }
}
```



The abstract class **Party** is not included in that list, only its two concrete subclasses **Individual** and **Organization**.



The important **Customer** subclass is missing in the V5.0.0 release. If needed, a Customer can be included as **PartyRole** instance, until it will be included in a later API version.

2.3.4. Historic Deviations

All **[Entity]RefOrValue** schemas that are used in some published V5 API strictly comply with the above pattern.

As mentioned, `PartyOrPartyRole` is an instance of this pattern, although its name does not strictly follow the naming convention. In a future release, it should be renamed to `PartyRefOrValueOrPartyRoleRefOrValue`, and all its occurrences in APIs should be checked if replacement by Ref-only pattern is possible.

2.4. Pattern for Related[Entity] schemas

2.4.1. Purpose

The `Related[Entity]` schema is used when it is necessary to refer another resource type from a resource representation **with a role**.

This is not used to describe a relation between instances of the same resource type.



The role describes from the perspective of the referring entity the semantics of the referred entity, not the semantics of the relationship.



`Related[Entity]` resource always is by reference.



For cases where it is necessary to include the related entity by value, `Related[Entity]RefOrValue` MUST be used.

The instantiation `RelatedEntity` of this pattern for the entity `Entity` of unspecified type MAY only be used where the type of the referred entity is not known at design time.

This should usually be only the case for general purpose structures that can be applied in many different contexts, like e.g. "Workflow", "Policy", "Intent".

At runtime, the actual type of the referred entity as indicated by the `@referredType` attribute of the `[Entity]Ref` instance held by the `entity` attribute MUST be more specific than `Entity`.



The `RelatedPartyOrPartyRole` schema is NOT an application of this pattern, as it contains a `PartyOrPartyRole`, which is via `oneOf` a `PartyRef` or `PartyRoleRef` OR a `Party` or `PartyRole` or any of their subclasses. The formally correct name would be `RelatedPartyRefOrValueOrPartyRoleRefOrValue`.

When inclusion by value is not required, the

`RelatedPartyRefOrPartyRoleRef` SHOULD be used instead, it contains a `PartyRefOrPartyRoleRef`, which is via `oneOf` a `PartyRef` or `PartyRoleRef`.

This confusing naming situation should be corrected in future releases.

2.4.2. Rules

- MUST include a mandatory attribute `[entity]` referring to `[Entity]Ref`.
- MUST include a mandatory attribute `role`.
- MUST NOT include any other attribute.

The type of the targeted entity is stated as `@referredType` in the `[Entity]Ref` instance held by the `[entity]` attribute.

Special Case `RelatedPartyOrPartyRole` schema:

Deviating from the above rule to use references, `RelatedPartyOrPartyRole` uses `PartyOrPartyRole` to realize the reference, which is via `oneOf` a `PartyRef` or `PartyRoleRef` OR a `Party` or `PartyRole` or any of their subclasses.



To avoid that the `Party` and `PartyRole` schemas, and all their sub-schemas are pulled in, use `RelatedPartyRefOrPartyRoleRef`, which uses the `PartyRefOrPartyRoleRef` schema to refer to a `Party` or `PartyRole` instance by reference, not by value.

`RelatedPartyRefOrPartyRoleRef` SHOULD be used whenever possible.

2.4.3. Examples

Example `RelatedPlace` schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "RelatedPlace.schema.json",
  "title": "RelatedPlace",
  "definitions": {
    "RelatedPlace": {
      "$id": "#RelatedPlace",
      "description": "A Place and an associated role as installation adress,
delivery address, etc....",
      "type": "object",
```

```

    "required" : [ "role", "place" ],
    "properties": {
      "role": {
        "type": "string",
        "description": "Role of the place, such as: [home delivery], [shop
retrieval])"
      },
      "place": {
        "$ref": "PlaceRef.schema.json#/definitions/PlaceRef"
      }
    }
  }
}

```



The above represents the intended structure of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword; in future releases, this reference will be removed.

Special case:

RelatedPartyRefOrPartyRoleRef:

Deviating from the naming schema, this schema is used to refer to a **Party** or **PartyRole** instance by reference, not by value. It contains a **PartyRefOrPartyRoleRef**, which is via **oneOf** a **PartyRef** or **PartyRoleRef**:

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "RelatedPartyRefOrPartyRoleRef.schema.json",
  "$privileged": true,
  "title": "RelatedPartyRefOrPartyRoleRef",
  "definitions": {
    "RelatedPartyRefOrPartyRoleRef": {
      "$id": "#RelatedPartyRefOrPartyRoleRef",
      "description": "RelatedParty reference. A related party defines party or
party role or its reference, linked to a specific entity",
      "type": "object",
      "properties": {
        "role": {
          "description": "Role played by the related party or party role in the
context of the specific entity it is linked to. Such as 'initiator', 'customer',
'salesAgent', 'user'",

```

```

        "type": "string"
      },
      "partyOrPartyRole": {
        "$ref":
"PartyRefOrPartyRoleRef.schema.json#/definitions/PartyRefOrPartyRoleRef"
      }
    },
    "required": [ "role" ]
  }
}

```



The above represents the intended structure of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword; in future releases, this reference will be removed.

2.4.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the [overview diagram](#)):

Containing [Entity] instead of [Entity]Ref

Related[Entity] is defined as reference to an entity with additional **role** attribute, where the reference is established by including an instance of **[Entity]Ref**.

In some cases an instance of the actual **[Entity]** schema is included instead.

This pulls the complete **[Entity]** schema including all its attributes into the schema of the referring **[Entity]**, which is not necessary if only a reference is needed and thus should be avoided whenever possible.

In other cases, the included instance is optional only, so that it is not ensured that the **Related[Entity]** refers to anything at all.

Changing the schemas to use the **[Entity]Ref** schema instead of the **[Entity]** schema highly impacts the structure defined by the resulting schema. It will create a breaking change for the listed existing V5 APIs and thus will not be realized.

For a future release, the concerned APIs should be analyzed in detail whether they have to include the entities by value.

In any case, the included instance must be corrected to be required.

Inheriting from Extensible

In all cases used by published V5 APIs, `Related[Entity]` is defined as `allOf Extensible`.

Because the `@type` attribute is required in `Extensible`, removing this reference removes the required attribute `@type`, which is a breaking change for consumers and thus can only be done in a future major version.

2.5. Pattern for `Related[Entity]RefOrValue` schemas

2.5.1. Purpose

The `Related[Entity]RefOrValue` schema is used when depending on the context it is necessary to either refer to the related entity, or include it by value.

`Related[Entity]RefOrValue` schema contains a role.



The role describes from the perspective of the referring entity the semantics of the referred entity, not the semantics of the relationship.



The instantiation `RelatedEntityRefOrValue` of this pattern for the entity `Entity` of unspecified type MAY only be used where the type of the referred entity is not known at design time. At runtime, the actual type of the entity provided by reference or by value MUST be more specific than `Entity`.



The `RelatedPartyOrPartyRole` schema is an application of this pattern, as it contains a `PartyOrPartyRole`, which is via `oneOf` a `PartyRef` or `PartyRoleRef` OR a `Party` or `PartyRole` or any of their subclasses. The formally correct name would be `RelatedPartyRefOrValueOrPartyRoleRefOrValue`.

When inclusion by value is not required, the `RelatedPartyRefOrPartyRoleRef` SHOULD be used instead, it contains a `PartyRefOrPartyRoleRef`, which is via `oneOf` a `PartyRef` or `PartyRoleRef`.

2.5.2. Rules

- MUST include a mandatory attribute `[entity]` referring to `[Entity]RefOrValue`.
- MUST include a mandatory attribute `role`.

- MUST NOT include any other attribute.

If the `[entity]` attribute is an `[Entity]Ref`, the `[Entity]Ref`s `@type`, `@baseType` and `@schemaLocation` attributes relate to the `[Entity]Ref` schema itself, and not to the targeted entity. The type of the targeted entity is stated as `@referredType` in the `[Entity]Ref` instance held by the `[entity]` attribute.

If the `[entity]` attribute is an actual `[Entity]`, the `@type`, `@baseType` and `@schemaLocation` attributes relate that `[Entity]` schema itself.

2.5.3. Examples

Example RelatedPlaceRefOrValue schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "RelatedPlaceRefOrValue.schema.json",
  "title": "RelatedPlaceRefOrValue",
  "definitions": {
    "RelatedPlaceRefOrValue": {
      "$id": "#RelatedPlace",
      "description": "A Place and an associated role as installation adress,
delivery address, etc....",
      "type": "object",
      "required" : [ "role", "place" ],
      "properties": {
        "role": {
          "type": "string",
          "description": "Role of the place, such as: [home delivery], [shop
retrieval])"
        },
        "place": {
          "$ref": "PlaceRefOrValue.schema.json#/definitions/PlaceRefOrValue"
        }
      }
    }
  }
}
```



The above represents the intended structure of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword; in future releases, this reference will be removed.

Special case RelatedPartyOrPartyRole schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "RelatedPartyOrPartyRole.schema.json",
  "$privileged": true,
  "title": "RelatedPartyOrPartyRole",
  "definitions": {
    "RelatedPartyOrPartyRole": {
      "$id": "#RelatedPartyOrPartyRole",
      "description": "RelatedParty reference. A related party defines party or party role or its reference, linked to a specific entity",
      "type": "object",
      "properties": {
        "role": {
          "description": "Role played by the related party or party role in the context of the specific entity it is linked to. Such as 'initiator', 'customer', 'salesAgent', 'user'",
          "type": "string"
        },
        "partyOrPartyRole": {
          "$ref": "PartyOrPartyRole.schema.json#/definitions/PartyOrPartyRole"
        }
      },
      "required": [ "role" ]
    }
  }
}
```



The above represents the intended structure of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword; in future releases, this reference will be removed.

2.5.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the [overview diagram](#)):

Attribute [entity] of type [Entity]RefOrValue only optional

[Entity]RefOrValue is defined to include a required attribute [entity] of type

`[Entity]RefOrValue`. In some cases, this attribute is not required.

Although changing the attribute to be required creates a breaking change for the existing V5 API, applying this change should be considered within V5. This change should be possible without impacting any real-world use of the concerned API, because using the schema without providing the actual link to the related entity, does not make much sense.

Inheriting from Extensible

In most cases used by published V5 APIs, `Related[Entity]RefOrValue` is defined as `allOf Extensible`.

Because the `@type` attribute is required in `Extensible`, removing this reference removes the required attribute `@type`, which is a breaking change for consumers and thus can only be done in a future major version.

2.6. Pattern for `[Entity]Relationship` schemas

2.6.1. Purpose

This schema **MUST** be used to describe a usually temporary relationship between independently existing instances of **same or different entity types**, for example to describe a relationship between `TroubleTicket`. This relationship is always typed, and often only valid for a specific period of time.



The `relationshipType` describes from the perspective of the referring entity the semantics of the relationship, not the semantics of the referred entity. If necessary, the relationship (not the referred entity) may be further qualified by additional information.



The instantiation `EntityRelationship` of this pattern for the entity `Entity` of unspecified type **MAY** only be used where the type of the referred entity is not known at design time. At runtime, the actual type of the referred entity as indicated by the `@referredType` attribute of the `[Entity]Ref` instance held by the `entity` attribute **MUST** be more specific than `Entity`.

2.6.2. Rules

- **MUST** include a mandatory attribute `[entity]` referring to `[Entity]Ref`.
- **MUST** include a mandatory attribute `relationshipType`.
- **MAY** include an attribute `validFor`.

- MAY include an array attribute `[entity]RelationshipCharacteristic` with array members referring to `Characteristic` schema.
- MUST NOT include any other attribute.

It is important to consider that the information provided in the `@type`, `@baseType` and `@schemaLocation` attributes relate to the `[Entity]Relationship` schema itself and not to the targeted entity.

The type of the targeted entity is stated as `@referredType` in the `[Entity]Ref` instance held by the `[entity]` attribute.

2.6.3. Examples

Example ResourceRelationship schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ResourceRelationship.schema.json",
  "title": "ResourceRelationship",
  "definitions": {
    "ResourceRelationship": {
      "$id": "#ResourceRelationship",
      "type": "object",
      "required": [ "resource", "relationshipType" ],
      "properties": {
        "relationshipType": {
          "type": "string"
        },
        "resource": {
          "$ref": "ResourceRef.schema.json#/definitions/ResourceRef"
        },
        "resourceRelationshipCharacteristic": {
          "type": "array",
          "items": {
            "$ref":
              "../Common/Characteristic.schema.json#/definitions/Characteristic"
          }
        }
      }
    }
  }
}
```



The above represents the intended structure of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword; in future releases, this reference will be removed.

Example of use of this schema for distinct resource type (a logical resource specification targeting a physical resource specification) but same base type (resource specification):

```
{
  "@type": "LogicalResourceSpecification",
  ...,
  "resourceSpecificationRelationship": [
    {
      "relationshipType": "dependency",
      "physicalResource": {
        "id": "42",
        "href": "https://mycsp.com:8080/tmf-
api/resourceCatalog/v5/physicalResourceSpecification/42",
        "@type": "PhysicalResourceSpecificationRef",
        "@referredType": "PhysicalResourceSpecification"
      }
    }
  ]
}
```



The above represents the intended structure of instances of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword, and in particular requires the **@type** attribute; in future releases, this reference will be removed, as shown above.

2.6.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the [overview diagram](#)):

Attribute [entity] of type [Entity]Ref not used

[Entity]Relationship is defined to include a reference to the targeted entity by using a required attribute **[entity]** of type **[Entity]Ref**.

In some cases, this attribute is missing, and the link is established by including some or all of the `[Entity]Ref` reference attributes – `id`, `href`, `name`, `@referredType` – directly into the `[Entity]Relationship` schema, or by inheriting them from `Entity` or `EntityRef`, or from some other `[Entity]Relationship` having the here described deviation.

Changing the schemas to use the `[Entity]Ref` schema instead of including the reference attributes directly highly impacts the structure defined by the resulting schema. It will create a breaking change for the listed existing V5 APIs and thus will not be realized in the current version.

In a future version, the pattern should be applied to all concerned schemas, and the concerned APIs should be modified accordingly.

Attribute `[entity]` of type `[Entity]RefOrValue`

In some cases, the attribute `[entity]` is of type `[Entity]RefOrValue` instead of `[Entity]Ref`.

Include the targeted entity by value instead of by reference is not in line with the declared semantics of the pattern, which is meant to establish a relationship between already existing entity instances.

Changing the schemas to use the `[Entity]Ref` schema instead of including the reference attributes directly impacts the structure defined by the resulting schema. It will create a breaking change for the listed existing V5 APIs and thus will not be realized in the current version.

In a future version, the pattern should be applied to all concerned schemas, and the concerned APIs should be modified accordingly.

Attribute `[entity]` optional

Although to be able to establish a relationship, the link to the targeted entity is necessary, in some cases the `[entity]` attribute is defined as optional.

Although changing the attribute to be required creates a breaking change for the existing V5 API, applying this change should be considered within V5. This change should be possible without impacting any real-world use of the concerned API, because using the schema without providing the actual link to the related entity, does not make much sense.

Inheriting from `[Entity]`

In some cases the `[Entity]Relationship` schema inherits from `[Entity]`, in addition to the included `[Entity]Ref` instance.

This in particular adds the attributes `id` and `href` of `[Addressable]`, which renders the

[Entity]Relationship to become a managed entity on its own, which in general should not be the case.

Changing the schema to not inherit from [Entity] impacts the structure defined by the resulting schema. It will create a breaking change for the listed existing V5 API and thus will not be realized in the current version.

Inheriting from Extensible

In some cases the [Entity]Relationship is defined as `allOf Extensible`.

Because the `@type` attribute is required in `Extensible`, removing this reference removes the required attribute `@type`, which is a breaking change for consumers and thus can only be done in a future major version.

Attribute relationshipType missing or optional

[Entity]Relationship is defined to be always typed, i.e. to include the required attribute `relationshipType` to describe the semantics of the relationship. In some cases, this attribute is missing or not required.

Adding the missing `relationshipType` attribute resp. changing it to be required establishes breaking changes to the related APIs. But it can be assumed that differentiating the relationship by its type is essential in any real-world use of the concerned APIs, so adapting the schemas should be considered nevertheless.

Containing attribute associationSpec

In some cases the [Entity]Relationship schema directly or indirectly via inheritance contains an attribute `associationSpec` of type `EntityRef`.

This deviation is found in the following schemas:

- EntityRelationship
- EntitySpecificationRelationship

For `EntityRelationship`, the attribute is optional and of type `AssociationRef` of type `EntityRef`; it is described as *"an EntityRef. Entity reference schema to be used for all entityRef class."* From this description, this attribute can be replaced by the missing `entity` attribute of type `EntityRef`.

For `EntitySpecificationRelationship`, the attribute is optional and of type `AssociationSpecificationRef`, which is a specialization of `EntityRef` without anything in addition. The attribute is described as *"A specification for an association used by this*

relationship". From this description, this attribute can be replaced by the missing **entitySpecification** attribute of type **EntitySpecification** and be made required.

These changes establish breaking changes to the related APIs, so this should be considered for a future version only.

Containing attribute role

In some cases, **[Entity]Relationship** contains the **role** attribute, which is not part of the pattern. The semantics of the **role** attribute is only specified for the **Related[Entity]** pattern and differentiates the semantics of the referred entity from the perspective of the referring entity. In contrast, the **[Entity]Relationship** is used to differentiate the semantics of a relationship between two entities. The **role** attribute is not necessary in this context and should be avoided.

Removing the **role** attribute from the concerned schemas establishes breaking changes to the related APIs and should be considered for a future version only.

Chapter 3. Patterns for concrete entity item schemas

Many entities managed by TMF Open APIs are composite entities, i.e. they consist of a number of constituent items that are managed as part of the composite entity. Examples for such composite entities are **ProductOrder**, **ShoppingCart**, **Quote**, etc, with respective items **ProductOrderItem**, **CartItem**, **QuoteItem**, etc.

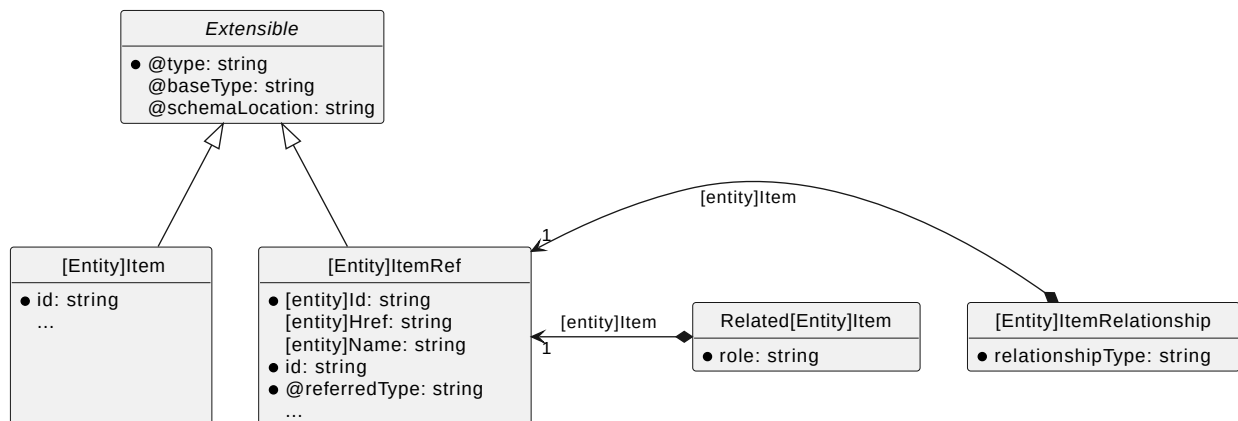
This chapter specifies patterns for building concrete entity item related schemas. They apply to all schemas explicitly named as "Item". Other constituent schemas may also follow these patterns, but may also deviate.

The term **[Entity]** represents the name of a concrete entity that is represented by a schema and managed through a TMF Open API, generally written with an uppercase initial letter.

The term **[entity]** represents the same name, written with a lowercase initial letter, e.g. used as leading part of an attribute name.

Overview

The patterns for item schemas are shown in the following diagram:



The patterns **[Entity]ItemRelationship** and **Related[Entity]Item** have been changed in V5 to use the **[Entity]ItemRef** pattern instead of either including the referring attributes **id**, **href** and **@referredType** directly or inheriting them from **EntityItemRef**, as was the case in prior versions. The changed structure enables custom extension of the patterns by inheritance independently of each other.

3.1. Pattern for [Entity]Item schemas

3.1.1. Purpose

The [Entity]Item schema is used to describe the representation of a constituent item of a composite entity API resource. An [Entity]Item is not managed as a distinct resource, but only as dependant sub-element of the corresponding [Entity].

Examples: `ProductOrderItem`, `CartItem`, `QuoteItem`, etc.



This pattern applies only to constituent items explicitly named "Item". Other constituent schemas may also follow this pattern, but are not considered here.



The [Entity]Item pattern MUST NOT be applied to the entity `Entity` of unspecified type, because for such an entity no knowledge exists on its structure, and thus not if it is a composite that has constituents.

3.1.2. Definition

[Entity]Item schema is a merge of `Extensible` schema and additional attributes.

3.1.3. Rules

- MUST include a direct or indirect (via recursion) reference to `Extensible` subschema (use of `allOf` keyword in [OpenAPI Specification 3.0.x \(OAS\)](#)).
- SHOULD include a mandatory attribute `id` which is unique in the scope of the composite [Entity] resource. If that attribute is not included, and the items of the composite are represented as an array, the array index can be used to identify the item within the composite, but it must be considered that the index may change if items are added or removed before the concerned item.
- SHOULD contain appropriate attributes of any of the basic datatypes `string`, `integer`, `number`, `boolean`, or sub-resources (JSON type `object`, more specifically defined by a schema), or arrays of such, used to describe the [Entity]Item class.

For specific attributes, the following additional rules apply:

- If the item needs a **type** attribute the attribute name MUST NOT be named `type` but `[entityItem]Type`.
- If the item requires a **timestamp** to manage the date and time when the item is created, this attribute MUST be named `creationDate` and be of type `string` with format

date-time.

- If the item requires a **validity period** to manage the time period during which the resource is valid, this attribute MUST be named **validFor** and MUST be a reference to class **TimePeriod**.
- If the item requires some **quantity value** to represent any property measured as integer or fractional multiple of some unit of measure, the corresponding attribute MUST be a reference to class **Quantity**.
- If the item requires some **monetary value**, the corresponding attribute MUST be a reference to class **Money**.
- If the item requires some **state value** to represent its condition within an operational process mainly controlled by automation, in particular if being prescriptive for what the item is allowed or able to do, having standardized transitions and business rules that can be formally described by a state machine, the attribute's name MUST either be **state** or end on **State** and be either of type **string** which MAY be restricted by specifying the available values as **enum**, or a reference to an enumeration class.
- If the item requires some **status value** to represent its stage in a business process, where the advancement depends on managerial business decisions, in particular when being descriptive as operational indicator or metric or reporting how well the item is currently doing, the corresponding attribute's name MUST either be **status** or end on **Status** and be either of type **string** which MAY be restricted by specifying the available values as **enum**, or a reference to an enumeration class.



The above distinction between **state** and **status** is not always clear-cut, and in some cases the terms are used interchangeably. The intention of the above rules is to establish a consistent usage throughout all TMF Open APIs and across other SDOs (Standards Developing Organization). Existing APIs may not yet fully comply with this intention, but it should be applied in future APIs and existing APIs should be aligned with this intention in future major releases.

3.1.4. Examples

CartItem schema



This is only a demonstrative example schema here, not the full CartItem schema, and also deviating from the actual CartItem schema used in the ShoppingCart API.

```
{
```

```

"$schema": "http://json-schema.org/draft-07/schema#",
"$id": "CartItem.schema.json",
"title": "CartItem",
"definitions": {
  "CartItem": {
    "$id": "#CartItem",
    "description": "CartItem schema as constituent of a ShoppingCart is used
for the temporary selection and reservation of a product offering in e-commerce,
call center and retail purchase. CartItem supports purchase of both physical and
digital goods and service (e.g. handset, telecom network service). CartItems may
contain a reference to the recipient (partyRole) or contact medium in case that
deviates from the Customer referred by the ShoppingCart, and the total items
price including promotions",
    "type": "object",
    "properties": {
      "id": {
        "type": "string",
        "description": "Unique identifier of the cart item, unique in scope of
the ShoppingCart"
      },
      "status": {
        "$ref":
"CartItemStatusType.schema.json#/definitions/CartItemStatusType",
        "description": "status of cart item. e.g \"active\" ,
\"savedForLater\"."
      },
      "itemPrice": {
        "$ref": "../Common/Money.schema.json#/definitions/Money",
        "description": "Amount of the shopping cart item, usually of money,
that represents the actual price paid by the Customer for the item"
      },
      "product": {
        "$ref": "../Product/Product.schema.json#/definitions/Product",
        "description": "the product selected by this cart item"
      },
      "quantity": {
        "$ref": "../Common/Quantity.schema.json#/definitions/Quantity",
        "description": "Quantity of cart item"
      },
      "relatedParty": {
        "type": "array",
        "items": {
          "$ref":
"../EngagedParty/RelatedPartyRefOrPartyRoleRef.schema.json#/definitions/RelatedPa
rtyRefOrPartyRoleRef"

```

```

    }
  }
},
"required": [ "id", "status", "product" ],
"allOf": [
  { "$ref": "../Common/EntityItem.schema.json#/definitions/Extensible" }
]
}
}
}

```

3.1.5. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the [\[patterns-for-concrete-schemas-items-only\]](#)):

Inherited from [Entity] instead of [Extensible]

As described earlier, this adds the attributes `id` and `href` of [\[Addressable\]](#), instead of adding the `id` attribute directly to the [\[Entity\]Item](#) schema, and thus establishes an implementation-inheritance which should be avoided.

This also could be misunderstood as indication that the item is a managed entity, which it is not. Although changing the schema to inherit from [Extensible](#) instead of from [\[Entity\]](#) impacts the structure defined by the resulting schema (it removes the `href`) and thus creates a breaking change for the listed existing V5 APIs, this change, in parallel to adding the `id` attribute directly to [\[Entity\]Item](#) if missing, should be considered.

Misuse of "Item" in schema name

A schema that nominally represents an `Item` actually is a Field transmitted as part of the header section of an HTTP message (request or response), and is not a constituent item of a composite entity. The name "Item" is misleading in this case and should be avoided.

Instead, the official term defined in [\[RFC9110\]](#) should be used, i.e. `HeaderField`.

Although renaming the schema establishes a breaking change to the related API, but should be considered to avoid misunderstandings.

3.2. Pattern for [Entity]ItemRef schemas

3.2.1. Purpose

`[Entity]ItemRef` schema is used to describe a reference to an existing item of an existing `[Entity]`.

`[Entity]ItemRef` SHOULD only be used when the semantics of the relation is clear from the context, in particular if there can be only one reference to an `[Entity]Item`, or if no semantic differentiation is required.



To differentiate the semantics of the relation, the `[Entity]ItemRelationship` pattern (which enforces a relationship type) SHOULD be used instead.

To differentiate the semantics of the referred item from the viewpoint of the referring entity, the `Related[Entity]Item` pattern (which enforces a role) SHOULD be used instead.

`[Entity]ItemRef` is used as part of `[Entity]ItemRelationship` and `Related[Entity]Item` to establish the link to the referred `[Entity]Item`.

It is recommended to use this schema to refer a distinct entity item type.



`[Entity]ItemRef` resource is always by reference.

The related entity item MUST NOT be included by value, therefore a pattern `[Entity]ItemRefOrValue` does not exist.

3.2.2. Rules

- Additional attributes MUST be added in the schema:
 - `[entity]Id` – required – unique identifier of the `[Entity]` to which the `[Entity]Item` belongs.
 - `[entity]Href` – optional – hyperlink (uri or uri-reference) to access the `[Entity]` to which the `[Entity]Item` belongs.
 - `itemId` – required – identifier of the `[Entity]Item`, unique in scope of the `[Entity]`.
 - `@referredType` – required – the actual type of the target instance.
- `[entity]Name` – optional – the `name` attribute of the `[Entity]` to which the `[Entity]Item` belongs.
- MUST NOT include any other attribute.

3.2.3. Examples

AgreementItemRef schema is defined to describe any AgreementItem reference:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "AgreementItemRef.schema.json",
  "title": "AgreementItemRef",
  "definitions": {
    "AgreementItemRef": {
      "$id": "#AgreementItemRef",
      "type": "object",
      "description": "It's a Agreement item that has been executed previously.",
      "properties": {
        "agreementId": {
          "type": "string",
          "description": "Unique identifier of a related entity."
        },
        "agreementHref": {
          "type": "string",
          "description": "Reference of the related entity."
        },
        "agreementName": {
          "type": "string",
          "description": "Name of the related entity."
        },
        "agreementItemId": {
          "type": "string",
          "description": "Id of an item of a Agreement"
        },
        "@referredType": {
          "type": "string",
          "description": "The actual type of the target instance when needed for disambiguation."
        }
      },
      "allOf": [
        { "$ref": "../Common/Extensible.schema.json#/definitions/Extensible" }
      ],
      "required": [
        "agreementId",
        "agreementItemId"
      ]
    }
  }
}
```

```
}
}
```

The above schema deviates from the pattern:



- The `@referredType` attribute is optional only.
- The `agreementItemId` attribute should be named `itemId` only.
- Inherits via `allof` from `Extensible`.

3.2.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the [overview diagram](#)):

Inherited from [Extensible]

In some cases, the `[Entity]ItemRef` schema inherits from `Extensible`, thus adding the attributes `@type` (required), `@baseType` and `@schemaLocation`.

Because the `@type` attribute is required in `Extensible`, removing this reference removes the required attribute `@type`, which is a breaking change for consumers and thus can only be done in a future major version.

@referredType optional only

Many `[Entity]ItemRef` schemas used in published V5 APIs define the `@referredType` attribute as optional only.

Changing `@referredType` to be required establishes a breaking change to the related APIs. It can be assumed that the actual type of the referred item is not essential in any existing real-world use of the concerned APIs, so the schemas should be adapted in a future version only.

Name mismatches of attributes [entity]Id, [entity]Href, itemId

Nearly all `[Entity]ItemRef` schemas used in published V5 APIs deviate from the naming of attributes `[entity]Id`, `[entity]Href`, `itemId` defined by the pattern.

Adapting the names establishes breaking changes to the related APIs and are only of cosmetic nature, so the schemas should be adapted in a future version only.

3.3. Pattern for Related[Entity]Item schemas

3.3.1. Purpose

The **Related[Entity]Item** schema is used when it is necessary to refer a constituent item of a composite entity from a resource representation **with a role**.



The role describes from the perspective of the referring entity the semantics of the referred entity, not the semantics of the relationship.



Related[Entity]Item resource is always by reference.

+ The related entity item **MUST NOT** be included by value, therefore a pattern **Related[Entity]ItemRefOrValue** does not exist.

3.3.2. Rules

- **MUST** include a mandatory attribute **[entity]Item** referring to **[Entity]ItemRef**.
- **MUST** include a mandatory attribute **role**.
- **MUST NOT** include any other attribute.

The type of the targeted entity is stated as **@referredType** in the **[Entity]ItemRef** instance held by the **[entity]Item** attribute.

3.3.3. Examples



The following example schema deviates from the pattern, see below.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "RelatedOrderItem.schema.json",
  "title": "RelatedOrderItem",
  "definitions": {
    "RelatedOrderItem": {
      "$id": "#RelatedOrderItem",
      "type": "object",
      "description": "It's a Order item that has been executed previously.",
      "properties": {
        "orderId": {
          "type": "string",
          "description": "Unique identifier of a related Order."
        }
      }
    }
  }
}
```

```

    "orderHref": {
      "type": "string",
      "description": "Reference of the related entity."
    },
    "orderItemId": {
      "type": "string",
      "description": "Id of an item of a product order"
    },
    "@referredType": {
      "type": "string",
      "description": "The actual type of the target instance when needed for
disambiguation."
    },
    "role": {
      "type": "string",
      "description": "Role played by the Order"
    },
    "orderItemAction": {
      "$ref":
"../Common/ItemActionType.schema.json#/definitions/ItemActionType",
      "description": "Action of the order item for this product."
    }
  },
  "required": [
    "role",
    "orderId",
    "orderItemId"
  ]
}
}
}

```



The above represents the intended structure of instances of the schema; the schema as used in V5 APIs additionally refers to the **Extensible** schema via the OAS 3 **allOf** keyword, and in particular requires the **@type** attribute; in future releases, this reference will be removed, as shown above.

3.3.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the

[overview diagram](#)):

Inherited from [Extensible]

In some cases, the `Related[Entity]Item` schema inherits from `Extensible`, thus adding the attributes `@type` (required), `@baseType` and `@schemaLocation`.

Because the `@type` attribute is required in `Extensible`, removing this reference removes the required attribute `@type`, which is a breaking change for consumers and thus can only be done in a future major version.

Not using [Entity]ItemRef

Most existing `Related[Entity]Item` schemas include the reference attributes – `id`, `href`, `name`, `@referredType` are included directly instead of using the `[Entity]ItemRef` schema to refer the targeted entity item.

Adjusting these schemas establishes breaking changes to the related APIs, so that the change will not be realized in the current version.

Attributes [entity]Id and/or itemId optional only

In some schemas, the attributes `[entity]Id` and `itemId` that are needed to establish the actual relation are optional only.

Although the change establishes a breaking change on the related API, using the `Related[Entity]Item` schema without providing the `[entity]Id` and `[entity]ItemId` attributes, i.e. the actual link to the related item, does not make much sense, this change should be applied in V5.

Attribute @referredType optional only

In nearly all `Related[Entity]Item` schemas used in published V5 APIs the `@referredType` attribute is optional only.

Changing `@referredType` to be required establishes a breaking change to the related APIs. It can be assumed that the actual type of the referred item is not essential in any existing real-world use of the concerned APIs, so the schemas should be adapted in a future version only.

role optional only

In some cases, the `role` attribute that is needed to differentiate the semantics of the referred item, is defined to be optional only.

The change to make **role** required establishes a breaking change to the related APIs, but as differentiating the semantics of the referred item is the main purpose of using **Related[Entity]Item** instead of **[Entity]ItemRef**, this change should be considered in V5.

3.4. Pattern for [Entity]ItemRelationship patterns

3.4.1. Purpose

An **[Entity]ItemRelationship** schema MUST be used to describe other than hierarchical relations of constituent items of the same composite entity, for example to describe the relationship between **ProductOrderItem**'s of a **'ProductOrder**.



The **relationshipType** describes from the perspective of the referring entity the semantics of the relationship, not the semantics of the referred entity.

3.4.2. Rules

- MUST include a mandatory attribute **[entity]Item** referring to **[Entity]ItemRef**.
- MUST include a mandatory attribute **relationshipType**.
- MUST NOT include any other attribute.

It is important to consider that the information provided in the **@type**, **@baseType** and **@schemaLocation** attributes inherited from **Extensible** relate to the **[Entity]ItemRelationship** schema itself and not to the targeted entity.

The type of the targeted entity is stated as **@referredType** in the **[Entity]ItemRef** instance held by the **[entity]Item** attribute.

3.4.3. Examples

CartItemRelationship:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "CartItemRelationship.schema.json",
  "title": "CartItemRelationship",
  "definitions": {
    "CartItemRelationship": {
      "$id": "#CartItemRelationship",
      "description": "Relationship among cart items mainly other than
        hierarchical relationships such as 'relyOn', 'dependentOn', 'shipping' etc.",
      "type": "object",
```

```

    "properties": {
      "id": {
        "type": "string",
        "description": "Unique identifier of the referred cart item"
      },
      "relationshipType": {
        "type": "string",
        "description": "Type of the cart item relationship"
      }
    },
    "allOf": [
      { "$ref": "../Common/Extensible.schema.json#/definitions/Extensible" }
    ],
    "required": [
      "id",
      "relationshipType"
    ]
  }
}

```



The above schema deviates from the pattern, see below.

3.4.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern (compare to the [overview diagram](#)):

Inherited from [Extensible]

In some cases, the [Entity]ItemRelationship schema directly or indirectly inherits from Extensible, thus adding the attributes @type (required), @baseType and @schemaLocation.

Because the @type attribute is required in Extensible, removing this reference removes the required attribute @type, which is a breaking change for consumers and thus can only be done in a future major version.

Not using [Entity]ItemRef

Most of the existing [Entity]ItemRelationship schemas include the attribute id directly to refer the targeted entity item instead of using the [Entity]ItemRef schema, which would also add the missing attributes @referredType (should be defined to be required) and href.

Adjusting these schemas establishes breaking changes to the related APIs and thus should be considered for a future version only.

Chapter 4. Event related schemas and patterns

Events are created in context of a given entity. Their schemas therefore name the concerned entity, followed by the type of the event which ends with the keyword "Event".

In the following, the term **[Event]** represents one of the event types defined for a given entity.

Common event types that SHOULD be supported for all entities are:

CreateEvent

The entity has been created.

AttributeValueChangeEvent

The values of one of more attributes of the entity has changed.

StateChangeEvent

(for entities having a **state** attribute only) some state of the entity has changed.

Note that entities can have more than one state attribute; in that case, the type of the state is contained in the attribute's name and must also be included in the event name.

StatusChangeEvent

(for entities having a **status** attribute only) some status of the entity has changed.

Note that entities can have more than one status attribute; in that case, the type of the status is contained in the attribute's name and must also be included in the event name.

DeleteEvent

The entity has ceased to exist.

For API implementations that realize the Version pattern (see part 4 of these guidelines), also the following events SHOULD be supported:

VersionCreateEvent

A new version of the entity has been created.

VersionCurrentEvent

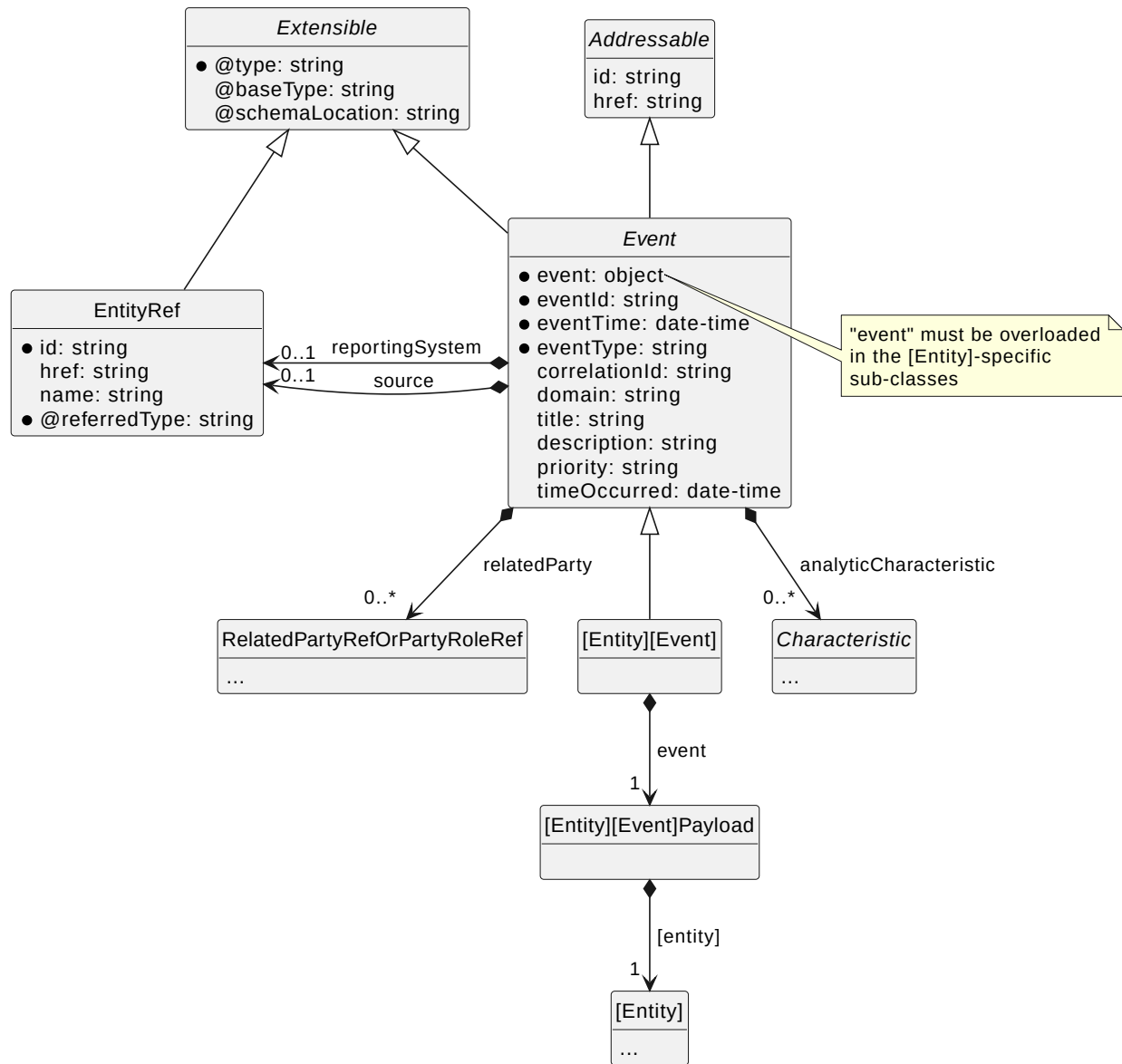
The entity version has been set to be the current version of the entity.

VersionDeleteEvent

The entity version has been deleted.

In addition to the standard event types described above, each API can define additional event types specific to the API.

The following diagram shows the structure of the classes defined by these schemas:



4.1. Event schema

4.1.1. Purpose

This schema is used for any event schema by extension, leveraging the TMF polymorphism pattern.

For details, see [part 1 of the TMF API Design Guidelines \(TMF630\)](#).

4.1.2. Definition

Event schema is a merge of **Addressable** and **Extensible** schemas, plus the following attributes:

- **correlationId**: The correlation id for this event.
- **domain**: The domain of the event.
- **title**: The title of the event.
- **description**: An explanation of the event.
- **priority**: Event priority.
- **timeOccurred**: The time the event occurred.
- **source**: Source Entity described by EntityRef.
- **reportingSystem**: Reporting System described by EntityRef.
- **relatedParty**: Collection of related parties.
- **analyticCharacteristic**: Collection of analytic characteristics.
- **eventId**: mandatory; The identifier of the notification as known by the sender.
- **eventTime**: mandatory; Time of the event occurrence.
- **eventType**: mandatory; The type of the notification.
- **event**: mandatory; The event linked to the involved resource object.

4.1.3. Rules

- **Event** schema must be used for any **[Entity][Event]** schema via **allOf** key word (see **[Entity][Event]** part).
- This schema is defined in the Common domain and MUST NOT be changed in any API.

4.1.4. Examples

Resources whose schemas extend the **Event** schema:

- **ProductCreateEvent**
- **ServiceOrderStateChangeEvent**
- **ShoppingCartAttributeValueChangeEvent**
- **PartyDeleteEvent**

4.1.5. Schema

The schema file is `schemas/Tmf/Common/Event.schema.json`

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "Event.schema.json",
  "title": "Event",
  "definitions": {
    "Event": {
      "$id": "#Event",
      "description": "event with common attributes.",
      "type": "object",
      "properties": {
        "correlationId": {
          "type": "string",
          "description": "The correlation id for this event."
        },
        "domain": {
          "type": "string",
          "description": "The domain of the event."
        },
        "title": {
          "type": "string",
          "description": "The title of the event."
        },
        "description": {
          "type": "string",
          "description": "An explanation of the event."
        },
        "priority": {
          "type": "string",
          "description": "A priority."
        },
        "timeOccurred": {
          "type": "string",
          "format": "date-time",
          "description": "The time the event occurred."
        },
        "source": {
          "$ref": "../Common/EntityRef.schema.json#/definitions/EntityRef",
          "description": "Source Entity described by EntityRef"
        },
        "reportingSystem": {
```

```

        "$ref": "../Common/EntityRef.schema.json#/definitions/EntityRef",
        "description": "Reporting System described by EntityRef"
    },
    "relatedParty": {
        "type": "array",
        "description": "Collection of related parties",
        "items": {
            "$ref":
"../EngagedParty/RelatedPartyRefOrPartyRoleRef.schema.json#/definitions/RelatedPa
rtyRefOrPartyRoleRef"
        }
    },
    "analyticCharacteristic": {
        "type": "array",
        "description": "Collection of analytic characteristics",
        "items": {
            "$ref":
"../Common/Characteristic.schema.json#/definitions/Characteristic"
        }
    },
    "eventId": {
        "type": "string",
        "description": "The identifier of the notification."
    },
    "eventTime": {
        "type": "string",
        "format": "date-time",
        "description": "Time of the event occurrence."
    },
    "eventType": {
        "type": "string",
        "description": "The type of the notification."
    },
    "event": {
        "description": "The event linked to the involved resource object",
        "type": "object"
    }
},
"allOf": [
    { "$ref": "Addressable.schema.json#/definitions/Addressable" },
    { "$ref": "Extensible.schema.json#/definitions/Extensible" }
],
"required": [
    "eventId",
    "eventTime",

```

```

        "eventType",
        "event"
    ]
  }
}
}

```



The above demonstrates the "should be" structure; the structure currently used for V5 APIs slightly deviates from this (without impact on the actual structure of runtime instances), see below.

4.1.6. Historic Deviations

The **Event** schema currently used in V5 APIs includes the **id** and **href** attributes directly instead of inheriting them from **Addressable**.

Adapting the schema to inherit from **Addressable** instead of including the attributes directly does not change the resulting structure of the actual events, so this change should be applied in V5.

TO DO:

It should also be checked if events should be handled as entities, i.e. be addressable at all, and the relation (if that exists) to the **eventId** attribute should be described.

TO DO:

The structure shown in Part 1, 11.5 deviates from the above, it contains an attribute **fieldPath** that is not contained in the Event schema found in the repository

4.2. Pattern for [Entity][Event] schemas

4.2.1. Purpose

The **[Entity][Event]** schema is used to describe the representation of any event of type **[Event]** related to the API resource **[Entity]**.

Common types of events are:

- **CreateEvent**
- **AttributeValueChangeEvent**
- **StateChangeEvent**
- **StatusChangeEvent**

- DeleteEvent
- VersionCreateEvent
- VersionCurrentEvent
- VersionDeleteEvent

Which of these event types are relevant for a given entity depends on the nature of the entity.

Other event types can be defined as needed.

4.2.2. Rules

- the name of the schema MUST be `[Entity][Event]`, where `[Entity]` is the name of the entity and `[Event]` is the type of the event, ending on the keyword "Event".
- MUST include a direct or indirect reference to `Event` schema (use of `allOf` keyword in `<<OAS3,OpenAPI Specification 3.0.x (OAS)>>`).
- MUST overload the `event` attribute of the `Event` base class, referring to the corresponding `[Entity][Event]Payload` schema.

4.2.3. Examples

The `CustomerDeleteEvent`:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "CustomerCreateEvent.schema.json",
  "title": "CustomerCreateEvent",
  "definitions": {
    "CustomerCreateEvent": {
      "$id": "#CustomerCreateEventEvent",
      "description": "CustomerCreateEvent generic structure",
      "required": [
        "event"
      ],
      "type": "object",
      "properties": {
        "event": {
          "$ref":
            "CustomerCreateEventPayload.schema.json#/definitions/CustomerCreateEventPayload"
        }
      }
    },
    "allOf": [
```

```

    { "$ref": "../../../Common/Event.schema.json#/definitions/Event" }
  ]
}
}
}

```

4.2.4. Historic Deviations

Not following the naming convention

Many event schemas do not follow the naming convention of ending in the "Event" keyword. This is in particular the case for state resp. status change events.

Modifying the schema names to follow the naming convention, i.e. to add the postfix "Event", establishes breaking changes to the related APIs, so that the change will not be realized in the current version.

4.3. Pattern for [Entity][Event]Payload schemas

4.3.1. Purpose

The [Entity][Event]Payload schema is used to describe the payload of any event of type [Event] related to the API resource [Entity].

4.3.2. Rules

- the name of the schema MUST be [Entity][Event]Payload, where [Entity] is the name of the entity and [Event] is the type of the event, ending on the keyword "Event".
- MUST contain an [entity] attribute, referring to the corresponding [Entity] schema.

4.3.3. Examples

The CustomerCreateEventPayload:

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "CustomerCreateEventPayload.schema.json",
  "title": "CustomerCreateEventPayload",
  "definitions": {
    "CustomerCreateEventPayload": {
      "$id": "#CustomerCreateEventPayload",
      "type": "object",

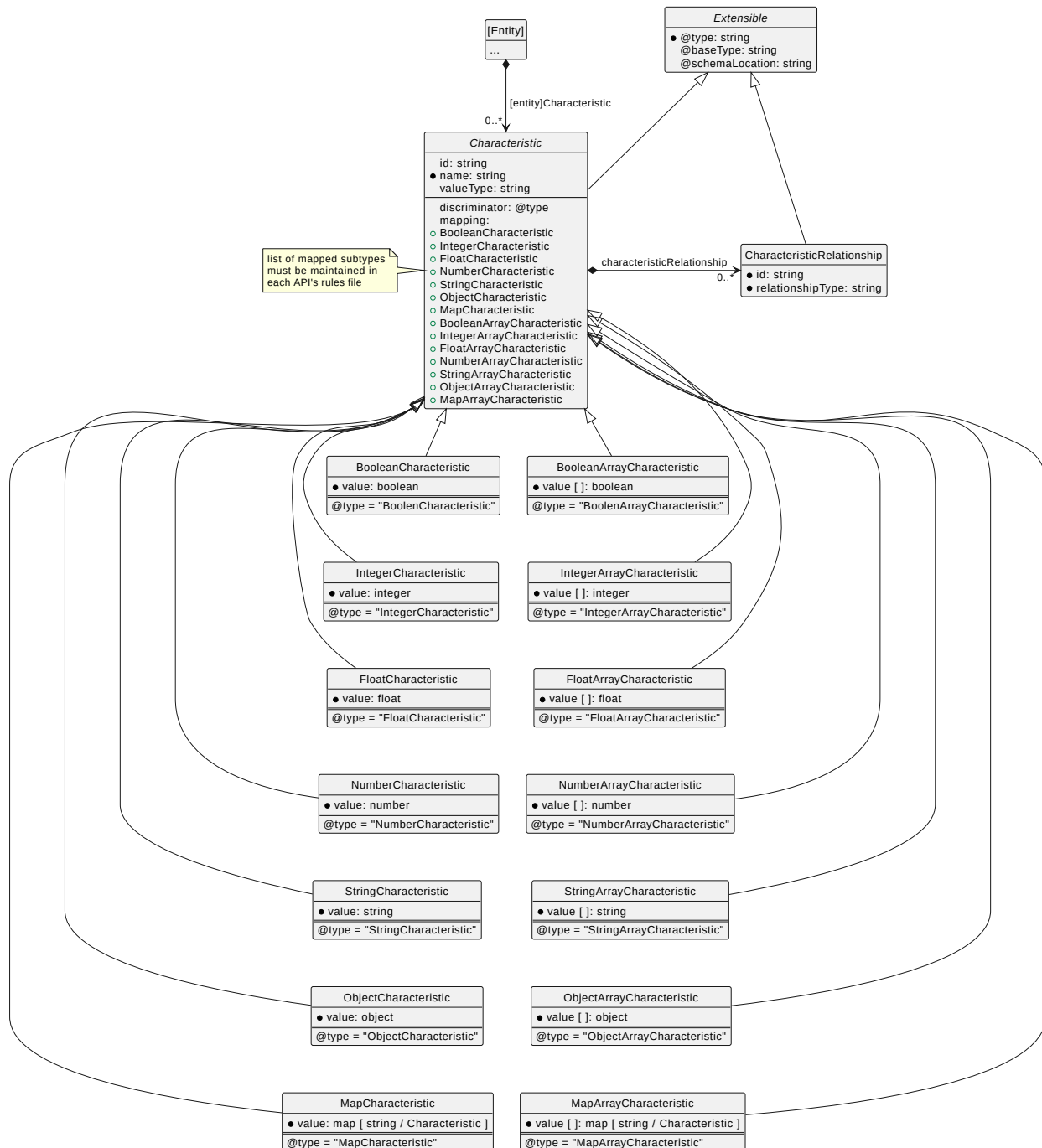
```

```
"description": "The event data structure",
"properties": {
  "customer": {
    "description": "The involved resource data for the event",
    "$ref": "../Customer.schema.json#/definitions/Customer"
  }
}
}
```

Chapter 5. Additional structural schemas

5.1. Characteristic schema

The following diagram shows the structure of the classes defined by these schemas:



5.1.1. Purpose

This schema including its type-specific realizations **MUST** be used in any schema describing the representation of an entity with characteristics.



A characteristic is essentially a name/value pair that describes a property of an entity or object.

Without any further information, a consumer retrieving some entity having characteristics will not be able to understand the semantics of the characteristic, i.e. what it means and how it is used, so consumer will not be able to use it in a meaningful way, without a private agreement between consumer and provider that is not part of an API contract.

Similarly, a producer of an entity will not know which characteristics to provide, and how to use them in a meaningful way.

We therefore **RECOMMEND** to add the characteristic pattern only to an **[Entity]**'s schema if there exists a corresponding **[Entity]Specification**, and only if that specification uses the **CharacteristicSpecification** and **CharacteristicValueSpecification** patterns to define the Characteristics required and allowed for the entity's instances.

This schema also allows defining relationship characteristic.

Characteristic is specialized to type-safe sub-classes, containing the **value** attribute having the respective type.

The schemas for the type-specific sub-classes **MUST** be used when a resource needs to manage characteristic.

Supported type-specific sub-classes are:

- **BooleanCharacteristic**
- **IntegerCharacteristic**
- **FloatCharacteristic**
- **NumberCharacteristic**
- **StringCharacteristic**
- **ObjectCharacteristic**
- **MapCharacteristic**

and corresponding `...ArrayCharacteristic`.

Characteristics of type `Quantity`, `TimePeriod`, `Money` or other "type-like" classes can be realized using the `ObjectCharacteristic` schema with `valueType` set to the name of that class.

A schema MUST NOT use its own characteristic schema, neither by creating its own extension of the here defined `Characteristic` schema nor by defining a separate schema, because that disables using the type-safe specializations.

5.1.2. Rules

- This schema and its specialized sub-class schemas are defined in the Common domain and MUST NOT be changed for any API.
- Any resource with the need to manage characteristic MUST use this schema and the appropriate type-specific specializations.
- In instantiations, `Characteristic` schema MUST NOT be used, but only the schemas for the type-safe sub-classes.
- The attribute of an `[Entity]` holding the entity's characteristic SHOULD be named `[entity]Characteristic` and be an array of `Characteristic`. It MUST NOT carry the Postfix "value".
- `Characteristic` pattern SHOULD only be used in conjunction with `CharacteristicSpecification` and `CharacteristicValueSpecification`

The `Any` datatype or any other schema that allows for undefined content is strongly discouraged and SHOULD NOT be used, because it precludes runtime validation and thus imposes security risks. Whenever possible, use an extension pattern with type-safe schemas instead.

5.1.3. Schema

Characteristic schema:

The schema file is `schemas/Tmf/Common/Characteristic.schema.json`

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "Characteristic.schema.json",
  "title": "Characteristic",
  "definitions": {
    "Characteristic": {
      "$id": "#Characteristic",
```

```

    "description": "Describes a given characteristic of an object or entity
    through a name/value pair. This is an abstract base class, the actual value is in
    one of the strongly-typed subclasses : StringCharacteristic,
    ObjectCharacteristic, FloatCharacteristic, BooleanCharacteristic,
    NumberCharacteristic, IntegerCharacteristic, StringArrayCharacteristic,
    ObjectArrayCharacteristic, BooleanArrayCharacteristic, NumberArrayCharacteristic,
    IntegerArrayCharacteristic...",
    "type": "object",
    "properties": {
      "id": {
        "type": "string",
        "description": "Unique identifier of the characteristic"
      },
      "name": {
        "type": "string",
        "description": "Name of the characteristic"
      },
      "valueType": {
        "type": "string",
        "description": "Data type of the value of the characteristic"
      },
      "characteristicRelationship": {
        "type": "array",
        "description": "Collection of characteristic relationships",
        "items": {
          "$ref":
"CharacteristicRelationship.schema.json#/definitions/CharacteristicRelationship"
        }
      },
      "allOf": [
        { "$ref": "Extensible.schema.json#/definitions/Extensible" }
      ],
      "required": [ "name" ]
    }
  }
}

```

Type-safe scalar sub-schema: [\[Type\]Characteristic](#)

The schema files are [schemas/Tmf/Common/\[Type\]Characteristic.schema.json](#)

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",

```

```

"$id": "[Type]Characteristic.schema.json",
"title": "[Type]Characteristic",
"definitions": {
  "BooleanCharacteristic": {
    "$id": "#[Type]Characteristic",
    "description": "A characteristic which value is a [type].",
    "required": [ "value" ],
    "type": "object",
    "properties": {
      "value": {
        "type": "[type]",
        "description": "Value of the characteristic"
      }
    },
    "allOf": [
      { "$ref": "Characteristic.schema.json#/definitions/Characteristic" }
    ]
  }
}

```

Type-safe array schema: `[Type]ArrayCharacteristic`

The schema files are `schemas/Tmf/Common/[Type]ArrayCharacteristic.schema.json`

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "[Type]ArrayCharacteristic.schema.json",
  "title": "[Type]ArrayCharacteristic",
  "definitions": {
    "[Type]ArrayCharacteristic": {
      "$id": "#[Type]ArrayCharacteristic",
      "description": "A characteristic which value is an array of [type].",
      "required": [ "value" ],
      "type": "object",
      "properties": {
        "value": {
          "type": "array",
          "description": "Collection of characteristic values",
          "items": {
            "type": "[type]",
            "description": "Characteristic value"
          }
        }
      }
    }
  }
}

```

```

    },
    "allOf": [
      { "$ref": "Characteristic.schema.json#/definitions/Characteristic" }
    ]
  }
}

```

Type-safe map schema: `MapCharacteristic`

The schema file is `schemas/Tmf/Common/MapCharacteristic.schema.json`

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "MapCharacteristic.schema.json",
  "title": "MapCharacteristic",
  "definitions": {
    "MapCharacteristic": {
      "$id": "#MapCharacteristic",
      "description": "A characteristic which value is a map of strings to values  
of any type.",
      "required": [ "value" ],
      "type": "object",
      "properties": {
        "value": {
          "type": "object",
          "description": "Collection of characteristic values",
          "additionalProperties": {
            "$ref": "Characteristic.schema.json#/definitions/Characteristic"
          }
        }
      }
    },
    "allOf": [
      { "$ref": "Characteristic.schema.json#/definitions/Characteristic" }
    ]
  }
}

```

The value is a map of strings (via [JSON Schema](#) 'additionalProperties' feature) to a Characteristic schema, which implies that any of the specialized sub-schemas can be referred.

Note that in order to include the schemas for the type-safe sub-classes it is necessary to list these schemas in the respective API's rules file. The tooling will then generate the appropriate discriminator and mappings in the Characteristic schema, e.g. (in YAML):

```
components:
  schemas:
    Characteristic:
      type: object
      properties:
        '@type':
          type: string
      ...
      required: [ '@type' ]
      discriminator:
        propertyName: '@type'
        mapping:
          BooleanCharacteristic:
            '#/components/schemas/BooleanCharacteristic'
          IntegerCharacteristic:
            '#/components/schemas/IntegerCharacteristic'
          FloatCharacteristic:
            '#/components/schemas/FloatCharacteristic'
          NumberCharacteristic:
            '#/components/schemas/NumberCharacteristic'
          StringCharacteristic:
            '#/components/schemas/StringCharacteristic'
          ObjectCharacteristic:
            '#/components/schemas/ObjectCharacteristic'
          MapCharacteristic:
            '#/components/schemas/MapCharacteristic'
          BooleanArrayCharacteristic:
            '#/components/schemas/BooleanArrayCharacteristic'
          IntegerArrayCharacteristic:
            '#/components/schemas/IntegerArrayCharacteristic'
          FloatArrayCharacteristic:
            '#/components/schemas/FloatArrayCharacteristic'
          NumberArrayCharacteristic:
            '#/components/schemas/NumberArrayCharacteristic'
          StringArrayCharacteristic:
            '#/components/schemas/StringArrayCharacteristic'
          ObjectArrayCharacteristic:
            '#/components/schemas/ObjectArrayCharacteristic'
          MapArrayCharacteristic:
            '#/components/schemas/MapArrayCharacteristic'
```

5.1.4. Examples

Schema example of an entity having characteristic:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "Service.schema.json",
  "title": "Service",
  "definitions": {
    "Product": {
      "$id": "#Service",
      "type": "object",
      "description": "...",
      "properties": {
        ...,
        "serviceCharacteristic": {
          "type": "array",
          "description": "Collection of service characteristics",
          "items": {
            "$ref":
"../Common/Characteristic.schema.json#/definitions/Characteristic"
          }
        },
        ...
      },
      "allOf": [
        { "$ref": "../Common/Entity.schema.json#/definitions/Entity" }
      ]
    }
  }
}
```



Although the array's name `serviceCharacteristic` is specific to the including entity `Service`, its elements are `Characteristic` instances and no specialized schema like e.g. `ServiceCharacteristic`.

5.1.5. Instantiation Examples

Instantiation example for a string characteristic:

```
"serviceCharacteristic": [
  {
    "id": "1gh-5gg",
    "@type": "StringCharacteristic",
    "@baseType": "Characteristic",
    "name": "TEL MSISDN",
```

```

    "valueType": "string",
    "value": "415 279 7439"
  }
]

```

Instantiation example for a map characteristic:



In this example a power cable belonging to e.g. a Router product is modelled as characteristic of a product; this is for illustrative purposes only, usually a cable will probably be modelled as a Product included as component or the Router product.

```

"productCharacteristic": [
{
  "id": "zzd-722",
  "@type": "MapCharacteristic",
  "@baseType": "Characteristic",
  "name": "PowerCable",
  "valueType": "string --> Characteristic",
  "value": {
    "powerPlugType": {
      "@type": "StringCharacteristic",
      "@baseType": "Characteristic",
      "name": "PowerPlugType",
      "value": "CEE-7/4"
    },
    "color": {
      "@type": "StringCharacteristic",
      "@baseType": "Characteristic",
      "name": "CableColor",
      "value": "black"
    },
    "length": {
      "@type": "ObjectCharacteristic",
      "@baseType": "Characteristic",
      "valueType": "Quantity",
      "value": {
        "amount": 2.5,
        "units": "m"
      }
    }
  }
}
]
}

```

```
]
```

Instantiation example for a "Quantity" characteristic:

```
"productCharacteristic": [  
  {  
    "id": "zzd-722",  
    "@type": "ObjectCharacteristic",  
    "@baseType": "Characteristic",  
    "name": "Length",  
    "valueType": "Quantity",  
    "value": {  
      "amount": 345.12,  
      "units": "cm"  
    }  
  }  
]
```

Instantiation example for an object characteristic:

```
"productCharacteristic": [  
  {  
    "id": "zzd-722",  
    "@type": "ObjectCharacteristic",  
    "@baseType": "Characteristic",  
    "name": "ElectricBikeConfiguration",  
    "valueType": "object",  
    "value": {  
      "@type": "ElectricBikeConfig",  
      "@schemaLocation": "http://serverlocation:port/electricBikeConfig.json",  
      "color": "red",  
      "motor": "CX Gen 4",  
      "size": "large",  
      "wheel size": 27.5  
    }  
  }  
]
```

5.1.6. Historic Deviations

Some resources used in already published V5 APIs use their own characteristic schema, either by inheriting from the generic **Characteristic** or by defining a separate realization of

characteristic. In both cases, taking advantage of the type-safe specialization pattern is not possible.

Although changing these schemas to use the generic **Characteristic** schema establishes breaking changes to the related APIs, this change should be applied in the current version.

5.2. CharacteristicSpecification schema

5.2.1. Purpose

This schema MUST be used when an entity definition specifies characteristics for instantiations of that entity. For example, **ProductSpecification** defines which characteristics will be populated when a **Product** is instantiated from that **ProductSpecification**.

It allows defining specification of relationship characteristic.

CharacteristicSpecification is NOT specialized to type-safe sub-classes, it is sufficient to sub-class the value specifications.

5.2.2. Rules

- This schema is defined in the Common domain and MUST NOT be changed in any API.
- Any resource with the need to manage specifications of characteristic MUST use this schema.
- The attribute holding the characteristic specifications SHOULD be named **[entity]CharacteristicSpecification** and MUST be an array of **CharacteristicSpecification**.
- Entity-specific specializations MUST NOT be created.

5.2.3. Schema

The schema file is **schemas/Tmf/Common/CharacteristicSpecification.schema.json**

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "CharacteristicSpecification.schema.json",
  "title": "CharacteristicSpecification",
  "definitions": {
    "CharacteristicSpecification": {
      "$id": "#CharacteristicSpecification",
      "description": "This class defines a characteristic specification.",
```

```
"type": "object",
"properties": {
  "id": {
    "type": "string",
    "description": "Unique ID for the characteristic specification"
  },
  "name": {
    "type": "string",
    "description": "A word, term, or phrase by which this characteristic
specification is known and distinguished from other characteristic
specifications."
  },
  "valueType": {
    "type": "string",
    "description": "A kind of value that the characteristic can take on,
such as numeric, text and so forth"
  },
  "description": {
    "type": "string",
    "description": "A narrative that explains the
CharacteristicSpecification."
  },
  "configurable": {
    "type": "boolean",
    "description": "If true, the Boolean indicates that the target
Characteristic is configurable"
  },
  "validFor": {
    "$ref": "TimePeriod.schema.json#/definitions/TimePeriod",
    "description": "The period of time for which a characteristic is
applicable."
  },
  "minCardinality": {
    "type": "integer",
    "description": "The minimum number of instances a Characteristic can
take on. For example, zero to five phone numbers in a group calling plan, where
zero is the value for the minCardinality."
  },
  "maxCardinality": {
    "type": "integer",
    "description": "The maximum number of instances a Characteristic can
take on. For example, zero to five phone numbers in a group calling plan, where
five is the value for the maxCardinality."
  },
  "isUnique": {
```

```

        "type": "boolean",
        "description": "Specifies if the value of this characteristic is unique
across all entities instantiated from the specification that uses this
characteristic. For example, consider a ProductSpecification for a set-top box,
with a CharacteristicSpecification cardID. Each set-top box must have a different
value for cardID, so this isUnique attribute would be set to true for the
characteristic."
    },
    "regex": {
        "type": "string",
        "description": "A rule or principle represented in regular expression
used to derive the value of a characteristic value."
    },
    "extensible": {
        "type": "boolean",
        "description": "An indicator that specifies that the values for the
characteristic can be extended by adding new values when instantiating a
characteristic for a resource."
    },
    "@valueSchemaLocation": {
        "type": "string",
        "description": "This (optional) field provides a link to the schema
describing the value type."
    },
    "charSpecRelationship": {
        "type": "array",
        "items": {
            "$ref":
"CharacteristicSpecificationRelationship.schema.json#/definitions/CharacteristicS
pecificationRelationship"
        },
        "description": "An aggregation, migration, substitution, dependency or
exclusivity relationship between/among Specification Characteristics."
    },
    "characteristicValueSpecification": {
        "type": "array",
        "items": {
            "$ref":
"CharacteristicValueSpecification.schema.json#/definitions/CharacteristicValueSpe
cification"
        },
        "description": "A CharacteristicValueSpecification object is used to
define a set of attributes, each of which can be assigned to a corresponding set
of attributes in a CharacteristicSpecification object. The values of the
attributes in the CharacteristicValueSpecification object describe the values of

```

the attributes that a corresponding Characteristic object can take on."

```

    }
  },
  "allOf": [
    { "$ref": "Extensible.schema.json#/definitions/Extensible" }
  ],
  "required": [ "name", "valueType" ]
}
}
}

```

5.2.4. Examples

Example for the use of CharacteristicSpecification for ServiceSpecification:



For backward compatibility, the attribute name `serviceSpecCharacteristic` is used instead of `serviceCharacteristicSpecification`.

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ServiceSpecification.schema.json",
  "title": "ServiceSpecification",
  "definitions": {
    "ServiceSpecification": {
      "$id": "#ServiceSpecification",
      "type": "object",
      "description": "ServiceSpecification is a class that ...",
      "properties": {
        ...,
        "serviceSpecCharacteristic" : {
          "type": "array",
          "items": {
            "$ref":
"../Common/CharacteristicSpecification.schema.json#/definitions/CharacteristicSpe
cification"
          }
        },
        ...
      },
      "allOf": [
        { "$ref": "../Common/Entity.schema.json#/definitions/Entity" }
      ]
    }
  }
}

```

```
}
}
```

5.2.5. Instantiation Examples

Example for JSON payload of CharacteristicSpecification from the above ServiceSpecification payload example:



For backward compatibility, the attribute name `serviceSpecCharacteristic` is used instead of `serviceCharacteristicSpecification`.

```
"serviceSpecCharacteristic": [
  {
    "id": "5843",
    "name": "operatingSystem",
    "description": "This characteristic describes the operating system running
the service",
    "valueType": "string",
    "@valueSchemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/StringCharacteristic.schema.json",
    "validFor": {
      "startDateTime": "2023-08-12T00:00:00.000Z",
      "endDateTime": "2026-03-07T00:00:00.000Z"
    },
    "configurable": true,
    "isUnique": false,
    "regex": "^[a-zA-Z0-9 +- ]+$",
    "extensible": false,
    "minCardinality": 0,
    "maxCardinality": 1,
    "characteristicValueSpecification": [
      {
        "value": "Android 13",
        "valueType": "string",
        "isDefault": true,
        "validFor": {
          "startDateTime": "2022-08-15T00:00:00.000Z",
          "endDateTime": "2026-03-07T00:00:00.000Z"
        },
        "@type": "StringCharacteristicValueSpecification",
        "@baseType": "CharacteristicValueSpecification",
        "@schemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/StringCharacteristicValueSpecification.schema.json"
```

```

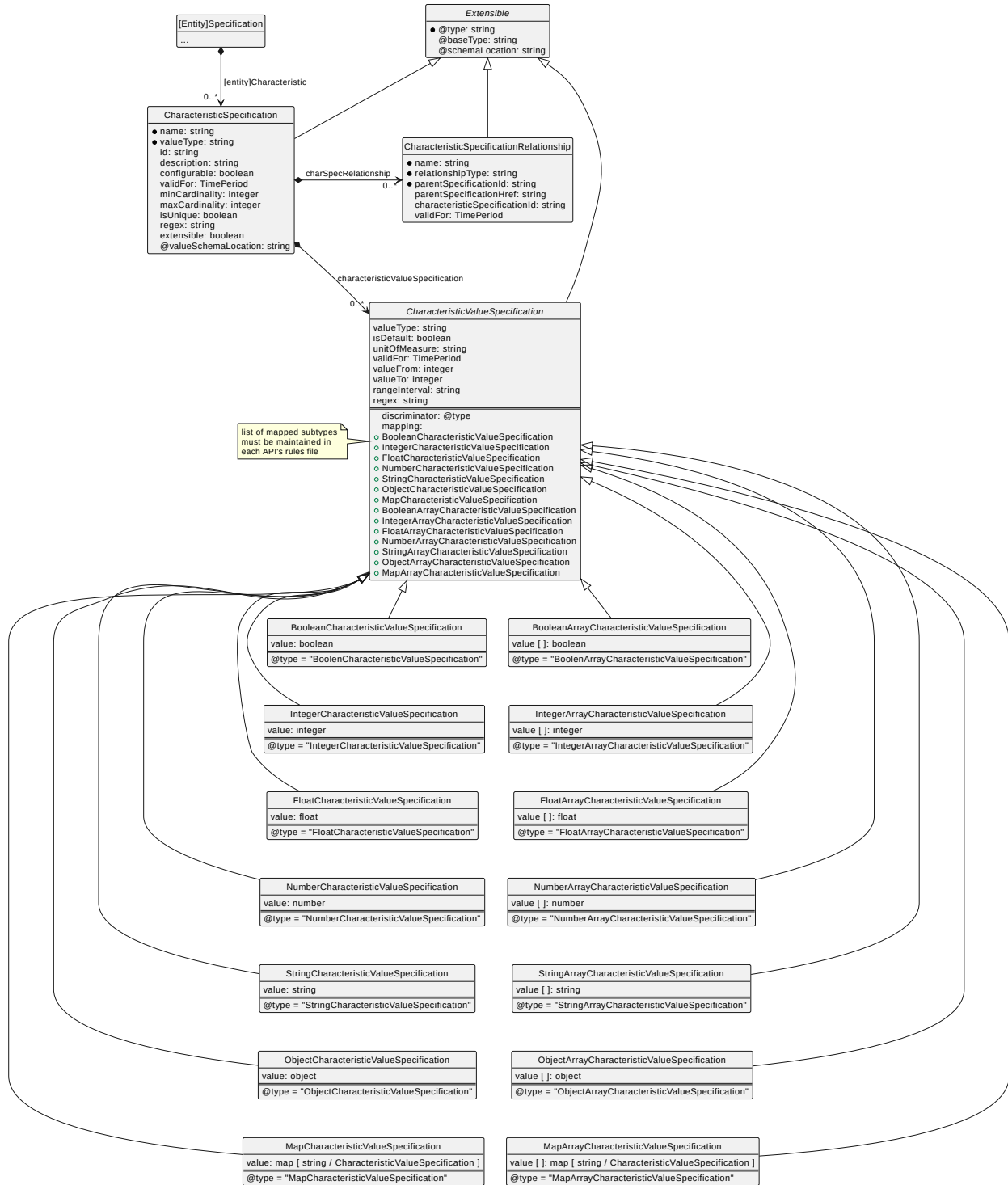
    },
    {
      "value": "Android 14",
      "valueType": "string",
      "isDefault": true,
      "validFor": {
        "startDateTime": "2023-10-04T00:00:00.000Z",
        "endDateTime": "2028-09-07T00:00:00.000Z"
      },
      "@type": "StringCharacteristicValueSpecification",
      "@baseType": "CharacteristicValueSpecification",
      "@schemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/StringCharacteristicValueSpecification.schema.json"
    }
  ],
  "charSpecRelationship": [
    {
      "relationshipType": "dependency",
      "parentSpecificationId": "1741",
      "parentSpecificationHref": "https://mycsp.com:8080/tmf-
api/serviceCatalogManagement/v5/serviceSpecification/1741",
      "characteristicSpecificationId": "4690",
      "name": "operatingSystem",
      "validFor": {
        "startDateTime": "2023-08-11T00:00:00.000Z",
        "endDateTime": "2026-03-07T00:00:00.000Z"
      },
      "@type": "CharacteristicSpecificationRelationship",
      "@schemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/CharacteristicSpecificationRelationship.schema.json"
    }
  ],
  "@type": "StringCharacteristicSpecification",
  "@baseType": "CharacteristicSpecification",
  "@schemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/StringCharacteristicSpecification.schema.json"
}
]

```

Note the use of type-safe **CharacteristicValueSpecification** schemas.

5.3. CharacteristicValueSpecification schema

The following diagram shows the structure of the classes defined by these schemas:



5.3.1. Purpose

This schema MUST be used in any schema describing the specification of type-specific characteristic values.

CharacteristicValueSpecification is specialized to type-safe sub-classes, containing the

value attribute having the respective type.

The schemas for the type-specific sub-classes **MUST** be used when a resource need to specify characteristic.

Supported type-specific sub-classes are:

- **BooleanCharacteristicValueSpecification**
- **IntegerCharacteristicValueSpecification**
- **FloatCharacteristicValueSpecification**
- **NumberCharacteristicValueSpecification**
- **StringCharacteristicValueSpecification**
- **ObjectCharacteristicValueSpecification**
- **MapCharacteristicValueSpecification**

and corresponding **ArrayCharacteristicValueSpecification**

CharacteristicValues of type **Quantity**, **TimePeriod**, **Money** or other "type-like" classes can be realized using the **ObjectCharacteristicValueSpecification** schema with **valueType** set to the name of that class.

5.3.2. Rules

- This schema and the type-safe specializations are defined in the Common domain and **MUST NOT** be changed for any API.
- Other schemas than **CharacteristicSpecification** **MUST NOT** use this schema, but only include it via their use of **CharacteristicSpecification**.

□ Exception: **ProductSpecificationCharacteristicValueUse**

The **Any** datatype or any other schema that allows for undefined content **MUST NOT** be used, because it precludes runtime validation and thus imposes security risks.

5.3.3. Schema

CharacteristicValueSpecification schema:

The schema file is **schemas/Tmf/Common/CharacteristicValueSpecification.schema.json**

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "CharacteristicValueSpecification.schema.json",
```

```

"title": "CharacteristicValueSpecification",
"definitions": {
  "CharacteristicValueSpecification": {
    "$id": "#CharacteristicValueSpecification",
    "description": "specification of a value (number or text or an object)
that can be assigned to a Characteristic. This is an abstract base class, the
actual value is in one of the strongly-typed subclasses :
BooleanArrayCharacteristicValueSpecification,
BooleanCharacteristicValueSpecification,
FloatArrayCharacteristicValueSpecification,
FloatCharacteristicValueSpecification,
IntegerArrayCharacteristicValueSpecification,
IntegerCharacteristicValueSpecification,
MapArrayCharacteristicValueSpecification, MapCharacteristicValueSpecification,
NumberArrayCharacteristicValueSpecification,
NumberCharacteristicValueSpecification,
ObjectArrayCharacteristicValueSpecification,
ObjectCharacteristicValueSpecification,
StringArrayCharacteristicValueSpecification,
StringCharacteristicValueSpecification...",
    "type": "object",
    "properties": {
      "valueType": {
        "type": "string",
        "description": "A kind of value that the characteristic value can take
on, such as numeric, text and so forth"
      },
      "isDefault": {
        "type": "boolean",
        "description": "If true, the Boolean Indicates if the value is the
default value for a characteristic"
      },
      "unitOfMeasure": {
        "type": "string",
        "description": "A length, surface, volume, dry measure, liquid
measure, money, weight, time, and the like. In general, a determinate quantity or
magnitude of the kind designated, taken as a standard of comparison for others of
the same kind, in assigning to them numerical values, as 1 foot, 1 yard, 1 mile,
1 square foot."
      },
      "validFor": {
        "$ref": "../Common/TimePeriod.schema.json#/definitions/TimePeriod",
        "description": "The period of time for which a value is applicable."
      },
      "valueFrom": {

```

```

        "type": "integer",
        "description": "The low range value that a characteristic can take on",
    },
    "valueTo": {
        "type": "integer",
        "description": "The upper range value that a characteristic can take
on"
    },
    "rangeInterval": {
        "type": "string",
        "description": "An indicator that specifies the inclusion or exclusion
of the valueFrom and valueTo attributes. If applicable, possible values are
\"open\", \"closed\", \"closedBottom\" and \"closedTop\"."
    },
    "regex": {
        "type": "string",
        "description": "A regular expression constraint for given value"
    }
},
"allOf": [
    { "$ref": "../Common/Extensible.schema.json#/definitions/Extensible" }
]
}
}
}

```



Characteristics that express a dimensional value, i.e. a number and unit, SHOULD NOT be modelled by a number type and the `unitOfMeasure` attribute, but by the `ObjectCharacteristicValueSpecification` schema with `valueType` set to `Quantity`, which is a class that has `amount` and `units` attributes.

Type-safe scalar sub-schema: `[Type]CharacteristicValueSpecification`:

The `[Type]CharacteristicValueSpecification` schema files are `schemas/Tmf/Common/[Type]CharacteristicValueSpecification.schema.json`

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "[Type]CharacteristicValueSpecification.schema.json",
  "title": "[Type]CharacteristicValueSpecification",
  "definitions": {
    "[Type]CharacteristicValueSpecification": {

```

```

    "$id": "#[Type]CharacteristicValueSpecification",
    "description": "A specification for a characteristic for which the value is
a [type]",
    "type": "object",
    "properties": {
      "value": {
        "type": "[type]",
        "description": "Value of the characteristic"
      }
    },
    "allOf": [
      { "$ref":
"CharacteristicValueSpecification.schema.json#/definitions/CharacteristicValueSpe
cification" }
    ]
  }
}

```

Type-safe array sub-schema: `[Type]ArrayCharacteristic`

The `schema` files are `schemas/Tmf/Common/[Type]ArrayCharacteristicValueSpecification.schema.json`

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "[Type]ArrayCharacteristicValueSpecification.schema.json",
  "title": "[Type]ArrayCharacteristicValueSpecification",
  "definitions": {
    "[Type]ArrayCharacteristicValueSpecification": {
      "$id": "#[Type]ArrayCharacteristicValueSpecification",
      "description": "A specification for a characteristic for which the value is
an array of [type]",
      "type": "object",
      "properties": {
        "value": {
          "type": "array",
          "description": "Value of the characteristic array members",
          "items": {
            "type": "[type]"
          }
        }
      }
    },
    "allOf": [

```

```

        { "$ref":
"CharacteristicValueSpecification.schema.json#/definitions/CharacteristicValueSpe
cification" }
      ]
    }
  }
}

```

Special case MapCharacteristicValueSpecification:

The schema file is [schemas/Tmf/Common/MapCharacteristicValueSpecification.schema.json](#)

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "MapCharacteristicValueSpecification.schema.json",
  "title": "MapCharacteristicValueSpecification",
  "definitions": {
    "MapCharacteristicValueSpecification": {
      "$id": "#MapCharacteristicValueSpecification",
      "description": "A specification for a characteristic for which the value is
a mapped object",
      "type": "object",
      "properties": {
        "value": {
          "type": "object",
          "additionalProperties": {
            "$ref":
"CharacteristicValueSpecification.schema.json#/definitions/CharacteristicValueSpe
cification"
          },
          "description": "Value of the characteristic"
        }
      },
      "allOf": [
        { "$ref":
"CharacteristicValueSpecification.schema.json#/definitions/CharacteristicValueSpe
cification" }
      ]
    }
  }
}

```

The value is a map of strings (via [JSON Schema](#) 'additionalProperties' feature) to a

CharacteristicValueSpecification schemas, which implies that any of the specialized sub-schemas can be referred.

Note that in order to include the schemas for the type-safe sub-classes it is necessary to list these schemas in the respective API's rules file. The tooling will then generate the appropriate discriminator and mappings in the CharacteristicValueSpecification schema, e.g. (in YAML):

```
components:
  schemas:
    CharacteristicValueSpecification:
      type: object
      properties:
        '@type':
          type: string
      ...
      required: [ '@type' ]
      discriminator:
        propertyName: '@type'
      mapping:
        BooleanCharacteristicValueSpecification:
          '#/components/schemas/BooleanCharacteristicValueSpecification'
        IntegerCharacteristicValueSpecification:
          '#/components/schemas/IntegerCharacteristicValueSpecification'
        FloatCharacteristicValueSpecification:
          '#/components/schemas/FloatCharacteristicValueSpecification'
        NumberCharacteristicValueSpecification:
          '#/components/schemas/NumberCharacteristicValueSpecification'
        StringCharacteristicValueSpecification:
          '#/components/schemas/StringCharacteristicValueSpecification'
        ObjectCharacteristicValueSpecification:
          '#/components/schemas/ObjectCharacteristicValueSpecification'
        MapCharacteristicValueSpecification:
          '#/components/schemas/MapCharacteristicValueSpecification'
        BooleanArrayCharacteristicValueSpecification:
          '#/components/schemas/BooleanArrayCharacteristicValueSpecification'
        IntegerArrayCharacteristicValueSpecification:
          '#/components/schemas/IntegerArrayCharacteristicValueSpecification'
        FloatArrayCharacteristicValueSpecification:
          '#/components/schemas/FloatArrayCharacteristicValueSpecification'
        NumberArrayCharacteristicValueSpecification:
          '#/components/schemas/NumberArrayCharacteristicValueSpecification'
        StringArrayCharacteristicValueSpecification:
          '#/components/schemas/StringArrayCharacteristicValueSpecification'
```

```

    ObjectArrayCharacteristicValueSpecification:
    '#/components/schemas/ObjectArrayCharacteristicValueSpecification'
    MapArrayCharacteristicValueSpecification:
    '#/components/schemas/MapArrayCharacteristicValueSpecification'

```

5.3.4. Instantiation Examples

Example for JSON payload of CharacteristicValueSpecification from ServiceSpecificationCharacteristic payload example:

```

"characteristicValueSpecification": [
  {
    "value": "Android 13",
    "valueType": "string",
    "isDefault": true,
    "validFor": {
      "startDateTime": "2022-08-15T00:00:00.000Z",
      "endDateTime": "2026-03-07T00:00:00.000Z"
    },
    "@type": "StringCharacteristicValueSpecification",
    "@baseType": "CharacteristicValueSpecification",
    "@schemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/StringCharacteristicValueSpecification.schema.json"
  },
  {
    "value": "Android 14",
    "valueType": "string",
    "isDefault": true,
    "validFor": {
      "startDateTime": "2023-10-04T00:00:00.000Z",
      "endDateTime": "2028-09-07T00:00:00.000Z"
    },
    "@type": "StringCharacteristicValueSpecification",
    "@baseType": "CharacteristicValueSpecification",
    "@schemaLocation": "https://mycsp.com:8080/tmf-
api/schema/Common/StringCharacteristicValueSpecification.schema.json"
  }
]

```

Example for JSON payload of MapCharacteristicValueSpecification:

```

"characteristicValueSpecification": [

```

```
{
  "value": {
    "someStringElement": "some string value",
    "someNestedMapElement" : {
      "firstNestedAttribute": "first value",
      "secondNestedAttribute": "second value"
    }
  },
  "valueType": "map",
  "validFor": {
    "startDateTime": "2022-08-15T00:00:00.000Z",
    "endDateTime": "2026-03-07T00:00:00.000Z"
  },
  "@type": "MapCharacteristicValueSpecification",
  "@baseType": "CharacteristicValueSpecification",
  "@schemaLocation": "https://mycsp.com:8080/tmf-api/schema/Common/MapCharacteristicValueSpecification.schema.json"
}
```

5.4. ExternalIdentifier schema

5.4.1. Purpose

An identification of an entity that is owned by or originates in a software system different from the current system, for example of a **ProductOrder** handed off from a business customer's ordering system or from a commerce platform into an order handling system. The structure identifies the system itself, the nature of the entity within the system (e.g. class name) and the unique ID of the entity within the system. It is anticipated that multiple external IDs can be held for a single entity, e.g. if the entity passed through multiple systems on the way to the current system. In this case the consumer is recommended by not required to sequence the IDs in the array in reverse order of provenance, i.e. most recent system first in the list.

5.4.2. Rules

- An **ExternalIdentifier** MUST include a reference to **Extensible** subschema (use of **allOf** key word in OpenAPI Specification 3.0.x (OAS))
- MUST include the following attribute:
 - **id** (mandatory)
- Additional attributes (optional)

□ owner

□ externalIdentifierType

- Schemas needing an external identifier MUST define an array attribute `externalIdentifier` whose array members refer to `ExternalIdentifier` schema.
- `ExternalIdentifier` MUST NOT be used to refer to any other entity than the entity it belongs to.



Via the general rule for `[Entity]Ref`, when an `[Entity]` schema defines the `externalIdentifier` array, its corresponding `[Entity]Ref` schema MAY also include the same `externalIdentifier` attribute.

5.4.3. Schema

The schema file is `schemas/Tmf/Common/ExternalIdentifier.schema.json`

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ExternalIdentifier.schema.json",
  "title": "ExternalIdentifier",
  "definitions": {
    "ExternalIdentifier": {
      "$id": "#ExternalIdentifier",
      "description": "An identification of an entity that is owned by or originates in a software system different from the current system, for example a ProductOrder handed off from a business customer's ordering system or from a commerce platform into an order handling system. The structure identifies the system itself, the nature of the entity within the system (e.g. class name) and the unique ID of the entity within the system. It is anticipated that multiple external IDs can be held for a single entity, e.g. if the entity passed through multiple systems on the way to the current system. In this case the consumer is expected to sequence the IDs in the array in reverse order of provenance, i.e. most recent system first in the list.",
      "type": "object",
      "required": [ "id" ],
      "properties": {
        "owner": {
          "type": "string",
          "description": "Name of the external system that owns the entity.",
          "examples": [ "MagentoCommerce" ]
        },
        "externalIdentifierType": {
          "type": "string",

```

```

        "description": "Type of the identification, typically would be the type
of the entity within the external system",
        "examples": [ "Order Number" ]
    },
    "id": {
        "type": "string",
        "description": "identification of the entity within the external
system.",
        "examples": [ "MC2255771199555" ]
    }
},
"allOf": [
    { "$ref": "../Common/Extensible.schema.json#/definitions/Extensible" }
]
}
}
}

```

Example of use of **ExternalIdentifier** schema:

```

"externalIdentifier": {
    "type": "array",
    "items": {
        "$ref":
"../Common/ExternalIdentifier.schema.json#/definitions/ExternalIdentifier"
    }
}

```

5.4.4. Historic Deviations

As explained in [Binding nature](#), some V5 APIs do not fully comply with the above rules.

The following describes typical deviations from the defined pattern:

Attribute name mismatch

In some cases, the array attribute that holds the external identifiers is named **externalId** or **externalReference** instead of **externalIdentifier**.

Adjusting the attributes' name establishes a breaking change for the related APIs, and does not improve the API's functionality. So the change is deferred to a later version.

Chapter 6. Additional structural patterns

6.1. Enumeration pattern

6.1.1. Purpose

This pattern is applied to describe that an attribute of an entity can take on its value only from a final set of enumerated values.

6.1.2. Rules

- This pattern SHOULD be used to describe the finite set of allowed values of an attribute.
- The name of the schema SHOULD refer to the semantics of the attribute whose values it defines, and SHOULD end with "Type".
- The enumerated values MUST be compatible with the indicated type.

6.1.3. Examples

Example schema for application of Type pattern:

E.g. to define the values for an attribute **action** of an OrderItem of some kind:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "ItemActionType.schema.json",
  "title": "ItemActionType",
  "definitions": {
    "ItemActionType": {
      "$id": "#ItemActionType",
      "description": "action to be performed on the entity managed by the item",
      "type": "string",
      "enum": [
        "add",
        "modify",
        "delete",
        "noChange"
      ]
    }
  }
}
```

6.1.4. Extensible Enumeration pattern

In some cases, the values that an attribute can take on are not strictly fixed, but can be supplemented by custom values; to formalize the list of predefined values, supplementary values can be enabled by applying the following "Extensible Enumeration" schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "[ExtensibleEnumeration]Type.schema.json",
  "title": "[ExtensibleEnumeration]Type",
  "definitions": {
    "[ExtensibleEnumeration]Type": {
      "$id": "#[ExtensibleEnumeration]",
      "description": "example of a schema that defines a list of discrete values,
but allows for additional values beyond the enumerated ones",
      "anyOf": [
        {
          "type": "string",
          "enum": [ "aaa", "bbb", "ccc" ]
        },
        {
          "type": "string"
        }
      ]
    }
  }
}
```



In the above, "[Extensible Enumeration]" is a placeholder for the actual name of the enumeration.

The values in the **enum** array are examples only.

An instance can take any of the enumerated values, or any other value. So this pattern can be seen as formalization of providing proposaly for values, instead of listing them in the attribute's description



The use of **anyOf** instead of **oneOf** is necessary, because if the instance takes on any of the enumerated values this validates not only the first alternative (the enum), but also the second one (the string). A **oneOf** required that exactly one of the alternatives be valid, and not both. In effect, using **oneOf** would allow all values but the enumerated

ones, so in fact is an "enumeration of exclusions".

Chapter 7. Type-like schemas

The following schemas represent extended data types that are used to model specific kinds of attributes.

Quantity	Money	TimePeriod
amount: float units: string	value: float unit: string	startDateTime: date-time endDateTime: date-time

They are defined in the Common domain and MUST NOT be changed in any API.

7.1. Quantity

A value measured by units, e.g. length, weight, volume, etc. It has an amount and a unit of measure.

Whenever possible, the units of measure defined by a standards body like [RFC3339](#) for duration units and the "[SI-Brochure](#)" of the [International System of Units \(SI\)](#) SHOULD be used.

7.2. Money

A monetary value, expressed in a specific currency. It has a numeric value and a unit attribute that holds the currency information.

Whenever possible, the currency codes defined by [\[ISO4217\]](#) SHOULD be used.

7.3. TimePeriod

A specific time interval with defined start and end. Missing start and/or end indicates an open-ended time period.

Chapter 8. Patterns and Schemas defined elsewhere in the Guidelines

For completeness, the following lists patterns and schemas defined in other parts of the Guidelines:

8.1. History

See [Part 1 of the Guidelines](#)

The history pattern allows implementations to expose the history and revision information on previous revisions of the resource(s) managed by a particular TM Forum API.

8.2. Version

See [Part 4 of the Guidelines](#)

In many business contexts, there is a requirement to allow entities to be versioned. The same entity may exist in several versions in parallel. For example, a "current" version is used in operational processes, whereas a "draft" version may already be in construction or planning, reflecting the future details of the entity; and likewise, access to older version from the past may still be necessary.

8.3. ImportJob / ExportJob

See [Part 4 of the Guidelines](#)

The common tasks of importing and exporting resource representations from/to a file, stream, or other data location.

8.4. TaskResource

See [Part 1 of the Guidelines](#)

If proper REST design is limited during translation from action-based operational interfaces to REST interfaces, it is allowable to define a resource for the operation and POST to that operation to execute.

8.5. Monitor

See [Part 1 of the Guidelines](#)

When a TaskResource representing a long-running operation is executed, it is often useful to be able to monitor the progress of the operation. The Monitor pattern allows implementations to expose the progress of a long-running operation.

8.6. Iterator

See [Part 1 of the Guidelines](#)

When a resource containing collections of sub-resources is too large to be returned in a single response, it is often useful to be able to iterate through the sub-resources in smaller chunks. The Iterator pattern allows implementations to expose sub-resource collections in smaller chunks, allowing clients to retrieve them in manageable pieces.

8.7. Error

The common data structure used to report details of errors that occurred before or during the processing of a request.

References

□ [ISO4217]

Currency codes

ISO (the International Organization for Standardization)

<https://www.iso.org/iso-4217-currency-codes.html>

□ [JSON Schema]

Draft 5: <https://datatracker.ietf.org/doc/html/draft-wright-json-schema-00>

Draft 7: <https://datatracker.ietf.org/doc/draft-handrews-json-schema/01>

□ [\[JSON Schema Validation\]](https://datatracker.ietf.org/doc/html/draft-wright-json-schema-validation-00)
 Draft 5: <https://datatracker.ietf.org/doc/html/draft-wright-json-schema-validation-00>
 Draft 7: <https://datatracker.ietf.org/doc/…​>

□ [OAS3]

OpenAPI Specification, Version 3

OpenAPI Initiative, The Linux Foundation

Version 3.0.0: <https://spec.openapis.org/oas/v3.0.0>

Version 3.0.1: <https://spec.openapis.org/oas/v3.0.1>

Version 3.0.2: <https://spec.openapis.org/oas/v3.0.2>

Version 3.0.3: <https://spec.openapis.org/oas/v3.0.3>

Version 3.0.4: <https://spec.openapis.org/oas/v3.0.4>

Version 3.1.0: <https://spec.openapis.org/oas/v3.1.0>

Version 3.1.1: <https://spec.openapis.org/oas/v3.1.1>

□ [RFC3339]

Date and Time on the Internet: Timestamps

IETF (Internet Engineering Task Force)

<https://www.ietf.org/rfc/rfc3339.html>

□ [SI]

SI-Brochure

International System of Units (SI)

<https://www.bipm.org/en/publications/si-brochure>

□ [TMF630-1]

TMForum

REST API Design Guidelines

Part 1: Practical guidelines for RESTful APIs naming, CRUD, filtering, tasks, history, notifications

□ [TMF630-2]

TMForum

REST API Design Guidelines

Part 2: Advanced guidelines for RESTful APIs polymorphism, extension patterns, depth and expand directive, entity RefOrValue

□ [TMF630-3]

TMForum

REST API Design Guidelines

Part 3: Guidelines for extending TMF Open API's with hypermedia support

□ [TMF630-4]

TMForum

REST API Design Guidelines

Part 4: Advanced guidelines for import and export tasks and for entity versioning

□ [TMF630-5]

TMForum

REST API Design Guidelines

Part 5: JSON patch extensions to manage arrays

□ [TMF630-6]

TMForum

REST API Design Guidelines

Part 6: JSON path extensions

□ [TMF630-7]

TMForum

REST API Design Guidelines

Part 7: JSON Schemas guideline

Administrative Appendix

This Appendix provides additional background material about the TM Forum and this document. In general, sections may be included or omitted as desired, however a Document History must always be included.

Document History

Version History

This section records the changes between this and the previous document version as it is edited by the team concerned. Note: this is an incremental number which does not have to match the release number and used for change control purposes only.

Version Number	Date Modified	Modified by:	Description of changes
5.0.0	10 July 2025	Lutz Bettge	Update from V4 to V5

Acknowledgments

Contributors to Prior Versions

Previous versions of this document were prepared by the following members of the TM Forum API team (in alphabetical order):

--	--	--

Contributors to Current Version

The following members of the TM Forum API team contributed to this version of the document (in alphabetical order):

Florin Tene	Cityfibre	Contributor
Jonathan Goldberg	TMForum	Contributor
Kamal Maghsoudlou	Ericsson	Contributor
Lutz Bettge	Deutsche Telekom	Contributor, Editor
Olivier Arnaud	Orange	Contributor